Sistemi operativi

Indice

1 Gestione dei processi e scheduling **5**

1.1 Sistema operativo 5

1.1.1 Obiettivi di un sistema operativo 5

1.1.2 Servizi offerti dal sistema operativo 5

1.1.3 Batch multiprogrammati 6

1.2 I processi 7

1.3 Gestione della memoria 8

1.4 Stato e descrizione dei processi 9

1.5 Creazione e terminazione dei processi 10

1.6 Modello a 2 stati 11

1.7 Modello a 5 stati 12

1.8 Context switch 12

1.9 Modalità di esecuzione dei processi 13

1.10 Creazione dei processi 14

1.11 Modello a 6 stati 15

1.12 Modello a 7 stati 16

1.13 Schedulazione 16

1.13.1 Scheduling della CPU nel dispatcher 19

1.13.2 Algoritmi di schedulazione 20

2 Thread **25**

2.1 Differenza tra thread e processi 25

2.2 Multithreading 26

2.3 Vantaggi e problemi dei thread 27

2.4 Stati dei thread 27

2.5 Operazioni di base 28

2.6 Esempi di multithreading 28

2.7 Categorie di threaed 29

2.7.1 User Level Thread . . . 30
2.7.2 Kernel Level Thread . . . 32
2.7.3 Approcci misti . . . 33
2.7.4 Relazione tra thread e processi . . . 34
2.8 Symmetric Multi Processing (SMP) . . . 34
2.9 Stati dei thread di Windows . . . 35
2.10 MicroKernel . . . 37
2.10.1 Vantaggi del microkernel . . . 37
2.10.2 Design del microkernel . . . 38
2.10.3 Funzioni minime del microkernel . . . 39

3 Gestione delle risorse e problematiche connesse 41
3.1 Grafo di allocazione delle risorse . . . 41
3.2 Stallo dei processi . . . 42
3.2.1 Condizioni per lo stallo . . . 42
3.2.2 Strategie per affrontare lo stallo . . . 43
3.3 Algoritmo dello struzzo . . . 43
3.4 Consentire lo stallo, rilevarlo e risolverlo . . . 43
3.4.1 Rilevare lo stallo . . . 44
3.4.2 Eliminare lo stallo . . . 46
3.5 Evitare lo stallo con politiche di allocazione . . . 48
3.5.1 Stato sicuro e insicuro . . . 48
3.5.2 Algoritmo del banchiere . . . 49
3.6 Impedire lo stallo rimuovendone le condizioni . . . 51
3.6.1 Negare la mutua esclusione . . . 51
3.6.2 Negare l'hold & wait . . . 52
3.6.3 Negare l'assenza di prerilascio . . . 52
3.6.4 Negare l'assenza circolare . . . 52
3.7 Concorrenza . . . 53
3.7.1 Terminologia della concorrenza . . . 53
3.7.2 Vantaggi e problemi della concorrenza . . . 54
3.7.3 Mutua esclusione . . . 55
3.7.4 Algoritmo di Dekker . . . 56
3.7.5 Supporti hardware . . . 59
3.7.6 Supporto del Sistema operativo e dei linguaggi di programmazione . . . 62

4 Gestione della memoria centrale **67**
4.1 Principi di base . . . 67
4.2 Allocazione della memoria . . . 67
4.3 Allocazione contigua . . . 68
4.3.1 Monoallocazione: Monitor Monoprocesso . . . 68
4.3.2 Partizionamento statico . . . 70
4.3.3 Partizionamento dinamico . . . 77
4.3.4 Segmentazione . . . 80
4.4 Allocazione non contigua . . . 85
4.4.1 Paginazione . . . 85
4.4.2 Memoria virtuale . . . 91

5 File system **101**
5.1 Struttura all'interno dei file . . . 102
5.2 Tipologie di file . . . 102
5.3 Formati dei file . . . 103
5.4 Accesso ai file . . . 103
5.5 Attributi dei file . . . 103
5.6 Operazioni sui file . . . 104
5.7 File mappati in memoria . . . 105
5.8 Directory . . . 105
5.9 Implementazione del file system . . . 107
5.9.1 Allocazione contigua . . . 107
5.9.2 Allocazione a lista concatenata . . . 107
5.9.3 Allocazione a lista concatenata con indice . . . 108
5.9.4 Allocazione mediante uso di tabelle i-node . . . 108
5.10 Struttura delle directory . . . 109
5.11 Gestione dello spazio su disco . . . 109
5.11.1 Dimensione dei blocchi . . . 109
5.11.2 Gestione dei blocchi liberi . . . 110
5.12 Affidabilità del file system . . . 110
5.12.1 Gestione dei blocchi fuori uso . . . 110
5.12.2 Copie di backup . . . 111
5.12.3 Consistenza del file system . . . 112
5.13 Prestazioni del file system . . . 113
5.13.1 Caching . . . 114
5.13.2 Riduzione dei movimenti del braccio del disco, soluzione soft . . . 114

5.13.3 Riduzione dei movimenti del braccio del disco, soluzione hard . . . 115
5.14 Modelli di protezione . . . 116
5.15 Dominio di protezione . . . 116
5.15.1 Strutture dati per la protezione . . . 117
5.16 Protezione nel sistema operativo UNIX . . . 117
5.17 Identificazione degli utenti . . . 118

Capitolo 1

Gestione dei processi e scheduling

1.1 Sistema operativo

1.1.1 Obiettivi di un sistema operativo

Gli **obiettivi di un Sistema operativo** sono:

- **convenienza** nell'uso del calcolatore rispetto ai potenziali utenti;
- **efficienza** nell'utilizzo del calcolatore e delle sue parti costitutive;
- **capacità** di evolversi rispetto a evoluzioni hardware, esigenze degli utenti e bug.

Il sistema operativo nasconde i dettagli hardware al programmatore e fornisce un'**interfaccia** per utilizzare il sistema, agisce quindi in maniera **trasparente**. Un'interfaccia è un componente fisico o logico che permette a due o più sistemi elettronici di comunicare e interagire.

1.1.2 Servizi offerti dal sistema operativo

I servizi offerti dal sistema operativo sono:

- **Creazione dei programmi**: compilatore, debugger come utilità offerte al programmatore che non fanno parte del SO ma sono accessibili tramite esso;

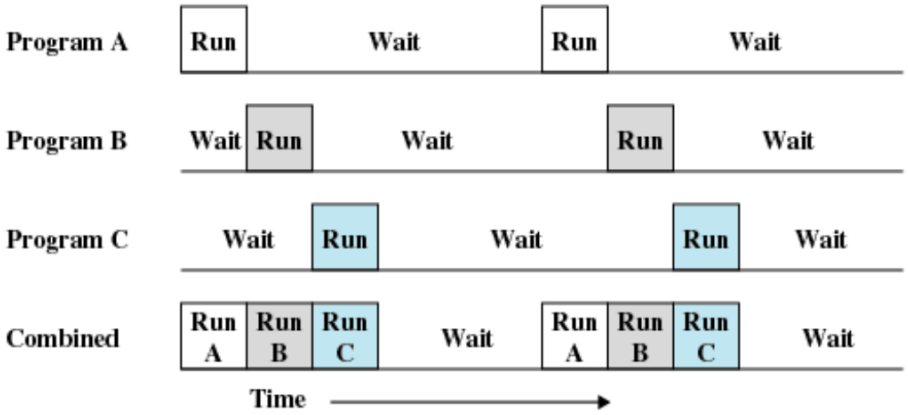
- **Esecuzione dei programmi:** caricamento in memoria dei programmi, inizializzazione dei dispositivi I/O ecc;
- **Accesso ai dispositivi I/O:** il programmatore ignora il set di istruzione e i segnali dei dispositivi;
- **Accesso controllato ai file:** comprensione del formato, meccanismi di protezione, associazione file indirizzi di memoria;
- **Accesso al sistema** inteso in senso lato;
- **Rilevazione e correzione degli errori** hardware o generati da programmi in esecuzione;
- **Contabilità e statistiche d'uso delle risorse**, dei tempi di risposta con il fine di migliorare le prestazioni.

Inoltre il SO dirige la CPU nell'utilizzo delle risorse del sistema e nella temporizzazione, cioè stabilire la sequenza dell'esecuzione dei programmi e decide quando un programma in esecuzione può utilizzare una risorsa (anche il processore è una risorsa).
Il **kernel** è la parte del sistema operativo che si trova in memoria centrale e contiene le funzioni del sistema operativo che vengono utilizzate frequentemente.

1.1.3 Batch multiprogrammati

I **Batch multiprogrammati** sono dei sistemi dove è consentita la multiprogrammazione, cioè l'esecuzione di più programmi in contemporanea.
Caratteristiche della multi-programmazione:

- presenza di più programmi in memoria;
- **Obiettivi:** limitare l'inattività del processore, quando un processo effettua un'operazione di I/O la CPU può essere utilizzata da un altro processo;
- elaborazione seriale dei task



(c) Multiprogramming with three programs

La **difficoltà della multi-programmazione** sta nel gestire la memoria e scegliere quale processo mandare in esecuzione.
Nella mono-programmazione un esempio può essere:

- Lettura di un record: 0.0015 sec;
- Esecuzione di 100 istruzioni: 0.0001 sec;
- Scrittura di un record: 0.0015 sec;
- Totale: 0.0031 sec

La percentuale di utilizzo della CPU è pari a $\frac{0.0001}{0.0031} = 0.032 = 3,2\%$ e notiamo quindi che è molto bassa.

1.2 I processi

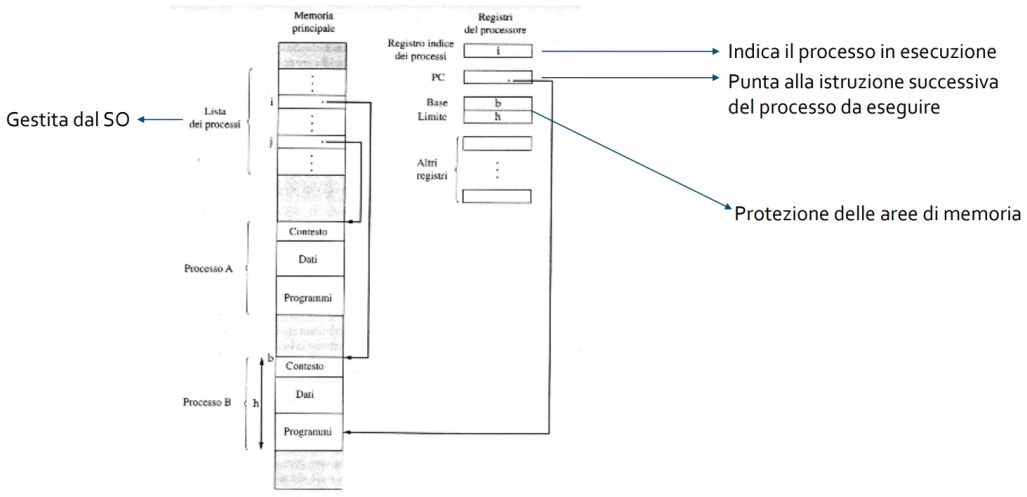
Un processo può essere chiamato in diversi modi, come job o task e la definizione di processo è:
un'attività caratterizzata dall'esecuzione di una sequenza di istruzioni, uno stato corrente e un set di istruzioni di sistema.
Le componenti di un processo sono:

- **programma**, cioè il codice eseguibile
- **Dati**

- variabili
- spazio di lavoro
- buffer

- **Contesto di esecuzione**, cioè delle informazioni necessarie al SO per gestire il processo
  - contenuto dei registri della CPU;
  - priorità;
  - stato di esecuzione;
  - stato di attesa su un dispositivo I/O.

Implementazione di un processo:



1.3 Gestione della memoria

Il sistema operativi deve compiere 5 compiti:

1. isolamento dei processi;
2. allocazione e gestione automatica della memoria: la gerarchia delle memoria deve essere trasparente all'utente;

3. supporto alla programmazione modulare: variazione di dimensione dei programmi

4. protezione e controllo dell'accesso: gestione di aree di memoria condivise tra i processi;

5. memorizzazione a lungo termine.

Queste necessità sono soddisfatte da:

- **memoria virtuale:** i programmi indirizzano la memoria con riferimenti logici, ignorando quelli fisici e quando un programma è in esecuzione solo una sua parte risiede in memoria centrale;

- **file system:** implementa la memorizzazione a lungo termine.

1.4 Stato e descrizione dei processi

Il compito principale del SO è quello di controllare l'esecuzione dei processi ed è possibile classificare lo stato attuale di un processo grazie, appunto, ad uno **stato**. In base allo stato di un processo questo viene gestito in maniera differente.
Il SO, quindi, ha bisogno di uno strumento per gestire i processi che tenga conto delle informazioni relativi ad essi.
Questo prende il nome di **Descrittore di un processo**, oppure **PCB**, Process Control Block.
Il Process Control Block è composto da:

- **Identificatore del processo**, un PID, Process Identification, che si tratta di un valore numerico;

- **Info sullo stato del processore**

  - Registri dati visibili all'utente che dipendono dall'architettura del calcolatore;
  - Registri di controllo e di stato:
    - Program Counter, PC e contiene l'indirizzo della prossima istruzione da eseguire;

        * Registri di stato che includono i flag per l'abilitazione degli interrupt;
        * Registri che contengono codici relativi alla condizione come segno, overflow ecc.
    - Puntatori allo stack che viene usato per procedure e funzioni

- **Informazioni di controllo del processo**
    - Schedulazione e informazioni di stato
        * stato del processo;
        * priorità nelle code di scheduling;
        * informazioni correlate alla schedulazione, come tempo di attesa, tempo di esecuzione ecc.;
        * se in attesa, l'evento per il quale è in attesa.
    - Strutturazione dati: puntatori ad altri processi
    - Comunicazione tra processi: flag, segnali e messaggi per la comunicazione
    - Privilegi in relazione all'uso della memoria, dei dispositivi ecc.
    - Gestione della memoria, cioè i limiti di memoria e gli indirizzi accessibili
    - Contabilizzazione delle risorse cioè le risorse controllate dal processo, come i dispositivi I/O, i file aperti ecc, e la loro storia

1.5 Creazione e terminazione dei processi

Gli eventi che portano alla **creazione** dei processi possono essere:

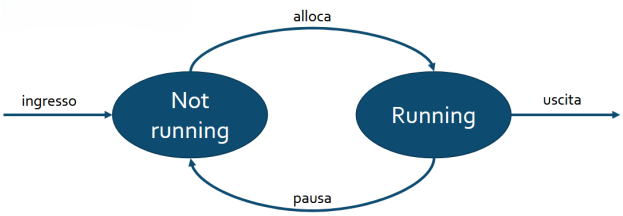
- richiesta dal terminale, cioè un utente accede al sistema
- il SO genera un processo sulla base della richiesta di un processo utente come ad esempio la stampa
- un processo utente genera un nuovo processo

Gli eventi che portano alla **terminazione** dei processi possono essere:

- terminazione normale;

- uscita dell'utente dall'applicazione;
- superamento del tempo massimo;
- memoria non disponibile;
- violazione dei limiti di memoria;
- fallimento di un'operazione;
- terminazione del genitore;
- richiesta del genitore.

1.6 Modello a 2 stati



Il modello a 2 stati è composto dagli stati di:

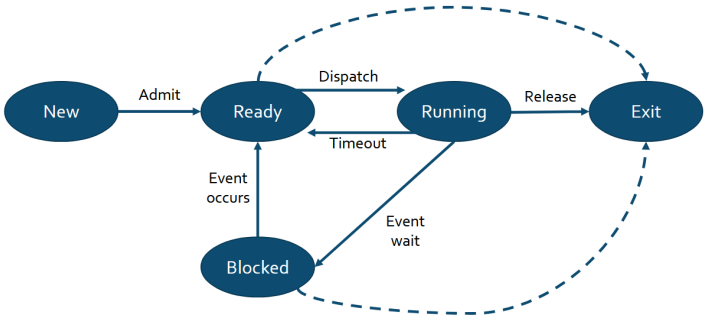
- *Running*, dove il processo sta eseguendo le istruzioni e sta utilizzando attivamente la CPU;
- *Not Running*, questo stato include due possibilità:
  - il processo è pronto per essere eseguito;
  - il processo è in attesa di un evento o di un dispositivo I/O.

Lo scheduler non può scegliere il processo da più tempo in attesa in quanto potrebbe essere in attesa di un trasferimento I/O.

1.7 Modello a 5 stati

Dal modello a 2 stati si passa al modello 5 stati:

- vengono introdotti gli stati *New* e *Exit* o *Terminated*;
- lo stato di *Not Running* viene diviso negli stati di *Ready*, cioè pronto per l'esecuzione e *Blocked* cioè in attesa di un evento, di una risorsa o altro.



Lo stato di *New* associa il PID, alloca e costruisce le tabelle per la gestione del processo ma non viene caricato in memoria.
Lo stato di *Exit* rilascia le risorse detenute dal processo e mantiene alcune informazioni, come le risorse utilizzate, utile per la contabilità.
Un processo può uscire dallo stato di *Running* per diversi motivi:

- è in attesa di un evento e quindi passa nello stato di *Blocked*;
- termina il tempo di esecuzione associato ad esso e passa nello stato di *Ready*.

1.8 Context switch

Il Context Switch riguarda il passaggio della CPU ad un nuovo processo e le cause possono essere:

- **Clock interrupt**, il processo ha terminato il tempo a sua disposizione e torna nella coda di ready

- **I/O interrupt**, un'operazione di I/O è terminata e il SO sposta il processo da blocked a ready e decide se riprendere l'esecuzione del processo precedente;
- **Memory fault**, l'indirizzo di memoria generato è sul disco e deve essere portato in RAM. Il SO carica il blocco, il processo che ha generato la richiesta è in blocked e al termine del trasferimento andrà in ready;
- **Trap**, errore di esecuzione e il processo potrebbe anche andare in exit;
- **Supervisor call**

Le operazioni svolte dal SO in modalità supervisor al momento del cambio del processo in stato di running sono:

- salvataggio del PCB del processo che abbandona la CPU;
- cambio di valore di stato nel PCB del processo, quindi da running può passare a blocked, ready o exit;
- Spostamento del PCB in nuova coda come ready o blocked, o deallocare le risorse in caso di exit;
- Aggiornamento delle strutture dati gestione memoria
- Selezione di un nuovo processo per lo stato di running;
- Aggiornamento del suo stato nel PCB;
- Ripristino del contesto.

Il context switch time è overhead cioè il sistema non svolge nessun compito utile all'utente e il tempo dipende dalla complessità del SO e dell'hardware.

1.9 Modalità di esecuzione dei processi

Modalità utente: esecuzione di processi utente
Modalità Sistema o Kernel o Controllo: esecuzione di istruzioni che hanno come scopo

- **Gestione dei Processi**

  - Creazione e terminazione
  - Schedulazione
  - Cambio di contesto
  - Sincronizzazione
  - PCB

- **Gestione della Memoria**
  - Allocazione
  - Trasferimento disco/RAM e viceversa
  - Gestione della paginazione, segmentazione...

- **Gestione I/O**
  - Gestione buffer
  - Allocazione canali I/O

- **Supporto**
  - Gestione interruzioni
  - Contabilità

1.10 Creazione dei processi

1. Assegnare al processo un PID unico; aggiungere una entry level alla tabella dei processi

2. Allocare lo spazio per il processo e per tutti gli elementi della sua immagine (PCB, User Stack, area di memoria dati e istruzioni, aree condivise)

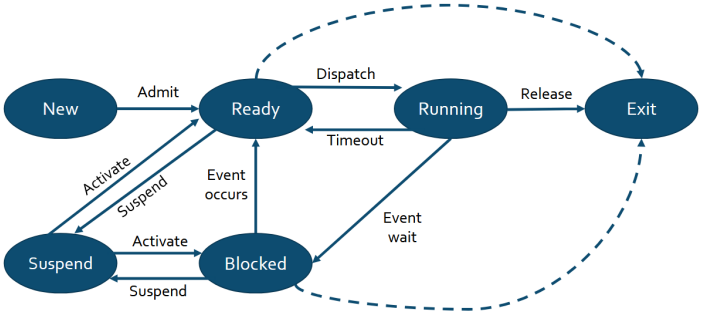
3. Inizializzazione del PCB
   - Stato del processore = 0
   - PC = prossima istruzione
   - Puntatori allo stack
   - Stato = ready

4. Inserimenti nella coda di ready

5. Estende le strutture al fine della fatturazione o delle statistiche

1.11 Modello a 6 stati

Dal modello a 5 stati si è passati al modello a 6 stati dove viene introdotto lo stato di *Suspend*.
Questo stato è stato introdotto in quanto con molta probabilità tutti i processi rimangono in attesa di eventi I/O e per questo è inefficiente tenere tutti i processi in attesa di un evento in memoria RAM e quindi, con lo stato di *Suspend* i processi in attesa di un evento vengono trasferiti dalla memoria RAM alla memoria secondaria, che è un'altra operazione di I/O ma è molto rapida.
Un'altra soluzione sarebbe quella di espandere la memoria ma è inefficiente in quanto costoso e perchè i programmi diventano sempre più grandi.



Il modello a 6 stati prevede due operazioni:

1. *Swap out*, la transizione del processo da *Blocked* a *Suspend* e quindi il passaggio dalla memoria RAM alla memoria secondaria;

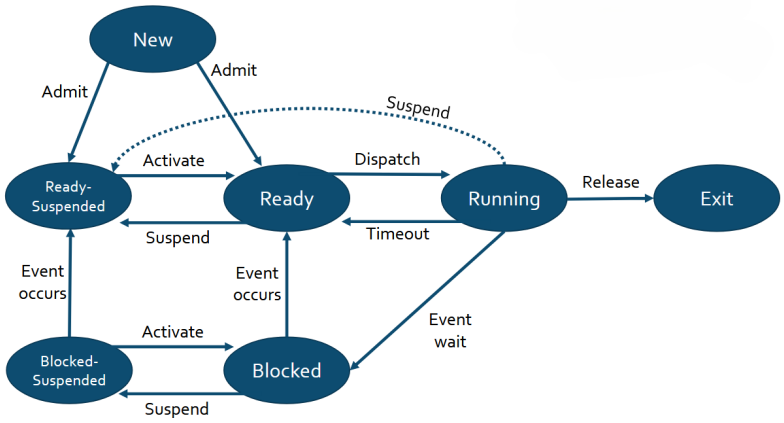
2. *Swap in*, ovvero la transizione del processo da *Suspend* a *Blocked* o *Ready* e quindi il passaggio dalla memoria secondaria alla memoria RAM.

Il modello a 6 stati pone lo stesso problema del modello a 2 stati, cioè non si può semplicemente caricare dalla memoria il processo da più tempo in attesa in quanto potrebbe ancora star aspettando che la risorsa necessaria si liberi o potrebbe ancora essere in attesa che l'evento desiderato si verifichi.
Pertanto si è passati al modello a 7 stati.

1.12 Modello a 7 stati

Nel modello a 7 stati, lo stato di *Suspended* viene suddiviso in due ulteriori stati:

- *Ready/Suspended* dove si trovano i processi in memoria secondaria ma pronti per passare in memoria RAM e per essere eseguiti;
- *Blocked/Suspended* dove si trovano i processi in memoria secondaria ancora in attesa che l'evento desiderato si verifichi o che la risorsa necessaria all'esecuzione diventi libera.



1.13 Schedulazione

La schedulazione, cioè la scelta dell'ordine di esecuzione dei processi, e la relativa politica di allocazione deve tenere in conto:

- **equità:** cioè tutti i processi appartenenti ad una stessa classe o con richieste simili o stesso costo devono avere la stessa possibilità di accesso alla risorsa;
- **tempo di risposta differenziale**: il SO discrimina tra classi che hanno bisogno di risorse diverse e di tempi diversi, ad esempio i processi I/O bound vengono schedulati per prima;
- **Efficienza:** massimizzare il throughput, cioè la quantità di dati trasmessi e minimizzare il tempo di risposta

Lo **scheduling** è insieme di tecniche e meccanismi interni del SO che amministrano l'ordine in cui il lavoro viene svolto e l'**obiettivo primario** dello scheduling è l'ottimizzazione delle prestazioni del sistema.
Il SO può prevedere fino a 3 tipi di scheduler:

- **Scheduler di lungo termine**, questo determina quali programmi sono ammessi nel sistema per essere processati cioè dallo stato di new allo stato di ready o ready-suspended. Il lavoro dell'SLT è quello di stimare il comportamento globale dei job in base a stime effettuate dal programmatore o dal sistema che forniscono informazioni sulle risorse necessarie, dimensione della memoria, tempo di esecuzione ecc.
  Le **strategie** principali dello scheduler di lungo termine sono:

  1. fornire alla coda dei processi pronti gruppi di processi che siano bilanciati tra loro nello sfruttamento della CPU e dell'I/O;
  2. aumentare il numero di processi provenienti dalla coda batch, quando il carico della CPU diminuisce;
  3. diminuire, fino anche a bloccare, i lavori provenienti dalla coda batch, quando il carico aumento e i tempi di risposta del sistema diminuiscono.

  La frequenza di chiamata dell'SLT è bassa e consente di implementare strategie complesse di selezione dei lavori e di dimensionamento del carico dei processi di inviare alla coda pronti.

- **Scheduler di medio termine**, questo si occupa di gestire la schedulazione delle transizioni da ready-suspended a ready e da blocked-suspended a blocked, quindi si occupa di trasferire i processi dalla memoria secondaria alla memoria centrale e viene attivato quando si rende disponibile lo spazio in memoria e quando l'arrivo di processi pronti scende al di sotto di una certa soglia specificata.
  La presenza di molti processi sospesi in memoria riduce la possibilità per nuovi processi pronti e in questo caso lo SBT è obbligato a scegliere trami pochi processi pronti.

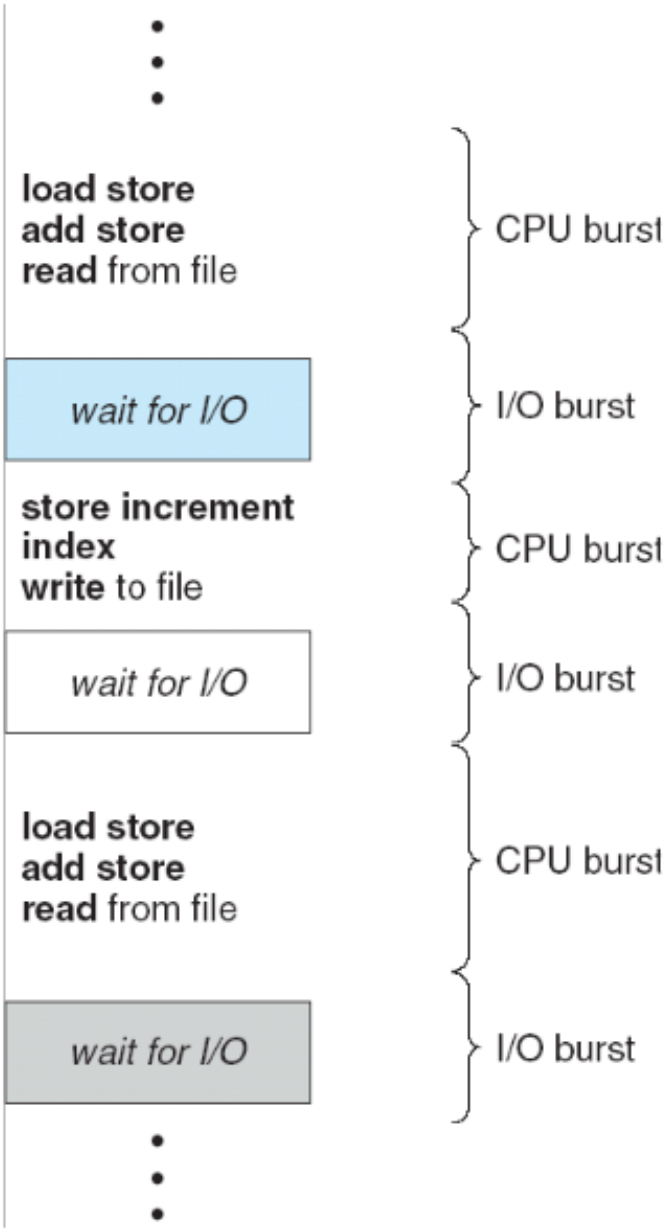
  – Utilizza le informazioni del PCB per stabilire la richiesta di memoria del processo;
  – tenta di allocare spazio in memoria centrale;

  - riposiziona il processo in memoria nella coda dei pronti.

- **Scheduler di breve termine** o anche detto **dispatcher** gestisce la transizione da ready a run e viene eseguito molto frequentemente e la sua **strategia** è quella di massimizzare le prestazioni del sistema secondo un specifico insieme di obiettivi.
  Questo viene invocato quando si verifica un evento:

  - clock interrupt;
  - I/O interrupt;
  - chiamate del SO;
  - signals.

1.13.1 Scheduling della CPU nel dispatcher



Lo scheduling della CPU riguarda la distribuzione delle sequenze di elaborazione della CPU e un processo durante la sua fase di esecuzione si può trovare in due stati:

1. Si trova nel suo ciclo di elaborazione e quindi sta utilizzando attivamente la CPU;
2. Si trova in attesa di un'operazione di I/O

Inoltre, i processi possono essere divisi in due macrocategorie:

- Processi **CPU bound** e sono processi che presentano poche operazioni di I/O e quindi sono più veloci;
- Processi **I/O bound** e sono processi che presentano molte operazioni di I/O e sono più lenti rispetto ai CPU bound.

1.13.2 Algoritmi di schedulazione

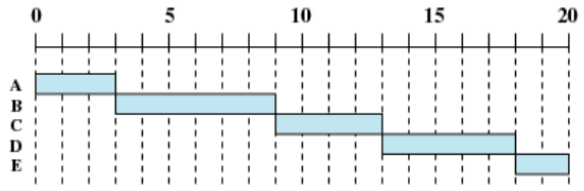
Definiamo due cose:

1. il **tempo di ricircolo** che rappresenta il tempo trascorso tra l'avvio di un processo, cioè la sua immissione nel sistema e la terminazione di esso;
2. il **tempo di attesa** cioè il tempo che un processo trascorre in attesa delle risorse. Si può calcolare come differenza tra il tempo di ricircolo e il tempo di esecuzione e quindi, valuta la sorgente di inefficienza che è il prezzo da pagare per condividere le risorse.

L'efficienza degli algoritmi di scheduling è misurabile utilizzando questi 2 tempi e un buon algoritmo di scheduling bilancia l'esecuzione dei processi al meglio, massimizzando l'uso del processore e riducendo i tempi di attesa.
Gli algoritmi possono avere 2 politiche:

- **non preemptive**, cioè non interrompibile e un processo in running abbandonerà questo stato solo se termina l'esecuzione o si blocca per un'operazione di I/O;
- **preemptive**, cioè interrompibile e un processo in running può essere interrotto e spostato in ready dal SO, e questo fa si che nessun processo può monopolizzare il processore, d'altra parte, però, crea dei problemi se ci sono dei processi che condividono dati e richiedono meccanismi di sincronizzazione

FCFS - First Come First Served



Nell'algoritmo FCFS si applica il principio della coda, ovvero ogni processo entra nella coda nello stato di *Ready* e quando un processo abbandona lo stato di *Running* si sceglie il processo che è da più tempo residente nello stato di *Ready*.
Questo algoritmo favorisce i processi CPU bound in quanto i processi I/O bound, che richiedono poco tempo di esecuzione, potrebbero attendere molto tempo prima che gli venga assegnata la CPU e quindi si genera l'effetto convoglio dove tutti i processi in coda aspettano che un processo CPU bound termini.
In questo algoritmo, quindi, le risorse del sistema non vengono sfruttate a pieno.

Algoritmo Event Driven

Questo algoritmo presenta uno schema che ragiona secondo un **valore di priorità** assegnato a ciascun processo e lo scheduler sceglierà semplicemente il processo pronto con priorità maggiore.
La priorità può essere assegnata dall'utente o dal sistema e può essere di tipo statico o dinamico. La priorità dinamica varia in base a:

- valore iniziale;
- caratteristiche del processo;
- richiesta di risorse;
- comportamento durante l'esecuzione.

Questo algoritmo viene applicato nei sistemi dove il tempo di risposta, soprattutto ad eventi esterni, è critico.
Il sistemista può influire sull'ordine di esecuzione dei processi modificando le

priorità assegnate ad essi e le prestazioni del sistema sono dipese dalla scelta delle priorità assegnate a ciascun processo.
Le priorità sono definite:

- esternamente al SO in base alla rilevanza del processo e alla sua criticità;
- internamente al SO utilizzando grandezze misurabili come l'uso della memoria, i file aperti e il rapporto tra picchi medi di I/O o CPU.

Questo algoritmo non è in grado di garantire il completamento di un processo in un intervallo di tempo finito dalla sua creazione, in quanto potrebbe essere continuamente sorpassato da processi a priorità più alta causando quindi la **starvation**.
La soluzione a questo problema è data dalla tecnica dell'**aging**, dove al passare del tempo di un processo in *Ready* la priorità del processo aumenta.

Algoritmo Round Robin - RR

Questo algoritmo utilizza il principio del **time slice** ovvero una preemption basata sul clock. Ogni processo utilizza il processore per un dato intervallo di tempo e al verificarsi di un interrupt il processo in esecuzione passa nello stato di *Ready*.
Con n processi in *Ready* e un tempo q assegnatogli, ogni processo ottiene $\frac{1}{n}$ del tempo di CPU in frazioni di tempo al più pari a q e il tempo massimo di attesa è di $(n-1)q$.
Le prestazioni dipendono dal quanto tempo assegnato perchè se la q è molto grande allora si degenera in un FCFS mentre se è troppo piccolo c'è un numero elevato di context switch e quindi si consumano molte risorse.
Questo algoritmo fornisce una buona condivisione delle risorse del sistema:

- i processi più brevi possono completare le loro operazioni in un buon q;
- i processi più lunghi sono forzati a passare più volte per la coda dei processi pronti;
- per i processi interattivi lunghi, se l'esecuzione riesce a completarsi in un q, il tempo di risposta è buono.

La realizzazione di un RR richiede il supporto di un Timer che invia un'interruzione ad ogni scadenza di q.

Highest Response Ratio Next - HRRN

Definiamo due variabili:

- w il tempo speso nella coda di *Ready*;
- s il tempo di servizio previsto.

Si definisce il Response Ratio come:

$$RR = \frac{w + s}{s}$$

Questo algoritmo manda in esecuzione il processo con il più alto valore di RR e quando un processo entra nella coda di *Ready* per la prima volta ha un RR = 1 e al passare del tempo nella coda di *Ready* il valore del Response Ratio aumenta e quindi viene implementa la tecnica dell'aging.

Shortest Process Next - SPN

In questo algoritmo, il processo scelto dalla coda di *Ready* è quello con il più breve tempo di esecuzione stimato.
Questo algoritmo è ottimale nel senso che fornisce il tempo media di attesa minimo ma è difficile stimare la durata della prossima sequenza di CPU ed è possibile che si verifichi la starvation per processi fortemente CPU bound. Questo algoritmo ha una versione preemptive dove se arriva un processo con una sequenza di CPU minore rispetto al processo attualmente in esecuzione si ha il rilascio della CPU a favore del processo appena arrivato. Questo schema viene chiamato *Shortest Remaining Time First (SRTF)* oppure *Shortest Remaining Time Next (SRTN).*

Schedulazione a code multiple

In questo schema la coda di Ready viene divisa in sotto-code:

- Foreground per i processi interattivi;
- Background per i processi batch

e ogni coda ha il proprio algoritmo di schedulazione ma è necessario uno scheduling fra le code che può essere:

- A priorità fissa e con prelazione, ad esempio serve prima la Foreground e poi la Background;
- Time slice dove ad ogni coda è associato un quanto di tempo di utilizzo della CPU.

È possibile un'ulteriore soluzione, quella della **Schedulazione a code multiple con feedback** dove viene implementata la tecnica dell'aging dove un processo può essere spostato da una coda all'altra.

Capitolo 2

Thread

Un thread è un **flusso di esecuzione indipendente, una traccia,** all'interno di un processo.
Le motivazioni per le quali solitamente un processo viene diviso in più thread sono:

- ottenere un parallelismo dei flussi di esecuzione all'interno del processo;
- gestire chiamate bloccanti o situazioni di risposta asincrona.

Ad esempio se dobbiamo implementare un web server, se questo fosse implementato come un monothread potremmo gestire solo un client alla volta e le altre richieste verrebbero ignorate e quindi ci sono due soluzioni possibili per risolvere questa situazione, cioè di implementare il server come un processo che resta in attesa delle richieste e poi scegliere se avviare un nuovo processo che processi la singola richiesta o avviare un thread che processi la singola richiesta.
Ovviamente, avviare ed effettuare un context switching di un thread è meno oneroso di avviare un processo.

2.1 Differenza tra thread e processi

Un **processo** possiede delle risorse:

- ha uno spazio di indirizzamento virtuale che contiene l'immagine del processo;

- può richiedere ulteriore memoria;
- può chiedere il controllo di canali di I/O;
- può chiedere il controllo di dispositivi;
- può chiedere files.

Il processo inoltre ha un'unità di esecuzione, quindi ha uno stato, una priorità e deve essere schedulato, tutte queste informazioni sono contenute nel PCB.
Un **thread** non ha risorse perchè utilizza quelle del processo e per questo viene anche chiamato **Light Weigth Process (LWP)**.
Un thread ha un proprio stato, una propria priorità e deve essere schedulato, ma tra i thread e sottostando alla schedulazione del processo. Tutte queste informazioni sono contenute nel **Thread Control Block (TCB)**.

2.2 Multithreading

Il multithreading è la capacità di un Sistema Operativo di supportare più thread per ogni processo.
Le diverse possibilità sono:

- **singolo processo a singolo thread**, come MS-DOS dove il thread esiste nel processo;
- **singolo processo e thread multipli**;
- **processi multipli a singolo thread**, come UNIX;
- **Multithread, più processi con più thread**, come Windows, MAC, Solaris ecc.

Sappiamo che un thread condivide le risorse del processo e in un sistema multithreading, i thread:

- condividono lo stato e le risorse del processo a cui appartengono;
- risiedono nello stesso spazio di indirizzamento,
- hanno accesso agli stati.

Dunque visto che i thread condividono tutti gli stessi dati è più facile scambiarsi informazioni.

2.3 Vantaggi e problemi dei thread

I thread presentano alcuni vantaggi:

- il **tempo di creazione** di un nuovo thread è minore del tempo di creazione di un nuovo processo;
- il **tempo di terminazione** di un thread è minore del tempo di terminazione di un processo;
- il **tempo di context switching** tra thread all'interno dello stesso processo è minore del tempo di context switching tra processi;
- i thread all'interno di uno stesso processo condividono memoria e files quindi c'è uno scambio dati senza richiedere l'intervento del kernel ma c'è la necessità di sincronizzare i thread nel momento in cui devono accedere alla stessa risorsa del processo.

Però, se un processo viene sospeso questo comporta la sospensione di tutti i thread contemporaneamente perchè si deve liberare spazio in memoria e i thread utilizzano lo stessso spazio di memoria condivisa. Inoltre, la terminazione di un processo richiede che tutti i thread siano terminati.

2.4 Stati dei thread

Gli stati dei thread, a differenza di quelli dei processi, sono solo 3:

- Ready;
- Running;
- Blocked.

Lo stato di suspended non c'è perchè non ha senso, in quanto se un processo viene swappato, di conseguenza anche i trhead a esso collegati vengono swappati.

2.5 Operazioni di base

Le operazioni che si possono effettuare su un thread sono:

- **Creazione**: quando viene creato un processo viene creato un thread e un thread può creare altri thread;
- **Blocco** a causa di un attesa di un evento e avviene il salvataggio del contesto per il thread;
- **Sblocco**: lo stato nel TCB viene modificato da blocked a ready e il thread viene accodato a quelli in attesa di processore;
- **Terminazione**: avviene la deallocazione del contesto registri e la deallocazione dello stack.

2.6 Esempi di multithreading

Il **Remote Procedure Call (RPC)** è la chiamata da parte di un processo a una procedura attiva su un elaboratore diverso dal chiamante.

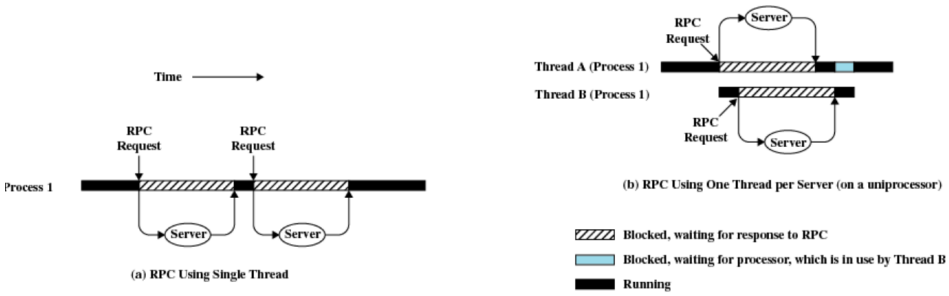
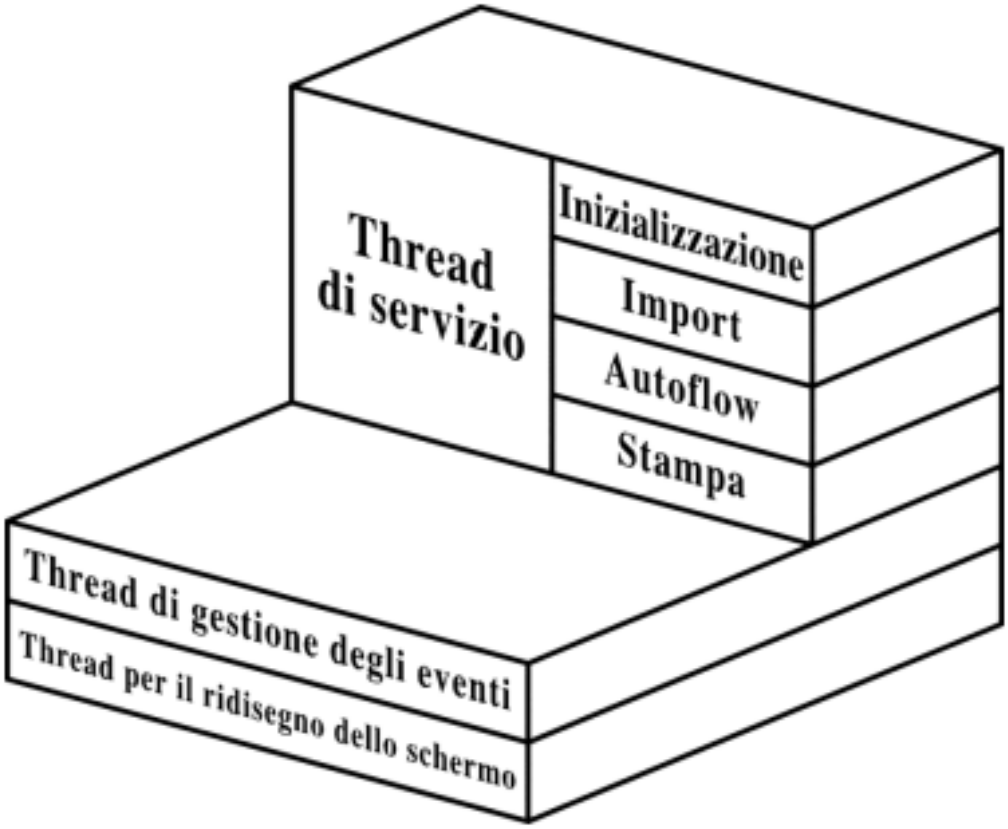


Figure 4.3 Remote Procedure Call (RPC) Using Threads

Vediamo un esempio di multithreading: un programma di video-scrittura, gestione e pubblicazione di pagine su desktop.
Ci sono tre thread sempre attivi:

1. Gestione degli eventi;
2. Gestione dei servizi come stampa, lettura dati, disposizione testo e attivazione di altri thread;

3. Disegno dello schermo.



Un altro esempio può essere lo scorrimento della pagina con barra laterale: qui abbiamo un thread eventi che controlla la barra di scorrimento e un thread di ridisegno dello schermo che ridisegna la pagina in base allo scorrimento. In questo esempio è necessario che i due thread siano sincronizzati.

2.7 Categorie di threaed

Esistono due tipologie di thread:

- **User Level Thread (ULT)**
  - **Realizzati tramite librerie senza l'intervento del kernel**

  - **Trasparenti al kernel**

  Lo svantaggio è che se il kernel è a singolo thread il blocco del thread blocca l'intero processo, ma il sistema operativo continua comunque a schedulare i processi,

- **Kernel Level Thread (KLT)**
  - **il kernel si occupa della creazione, scheduling e gestione**;
  - **possono essere eseguiti su diversi processori**;

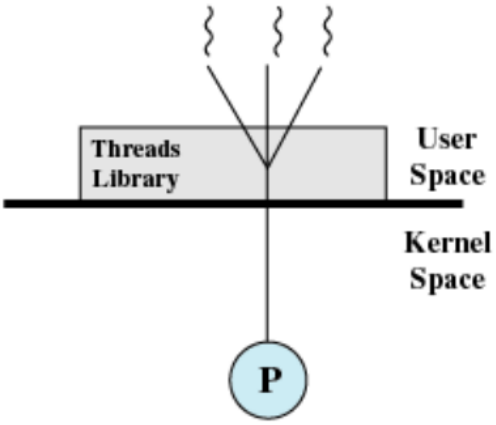
  Questa tipologia di thread hanno una gestione più lenta degli ULT.

2.7.1 User Level Thread

Come abbiamo detto questi thread sono realizzati tramite delle libreria senza l'intervento da parte del kernel quindi il lavoro di gestione dei threads è svolto dalla libreria utente e il kernel ignora la loro esistenza. Il modello di questi thread è **molti a uno**.
La libreria permette:

- creazione e distruzione dei threads;
- scambio messaggi tra threads;
- schedulazione;
- salvataggio e caricamento dei contesti dei thread.

Bisogna ricordare che tutte queste attività vengono **svolte all'interno del processo utente**, quindi il **kernel continua a schedulare i processi come unità a se stanti.**



Vantaggi e svantaggi I **vantaggi** dei thread ULT sono:

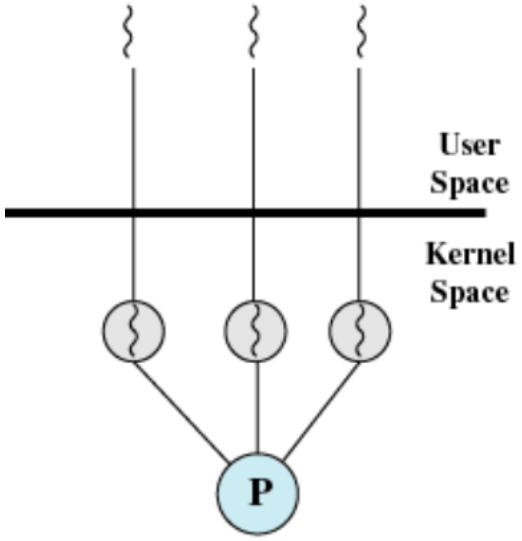
- **risparmio di sovraccarico**, in quanto il cambio di thread avviene all'interno dello spazio di indirizzamento utente e non viene richiesto l'intervento del kernel;
- **Schedulazione diversa per ogni applicazione**: questi thread sono ottimizzati in base al tipo di applicazione;
- **ULT eseguito da qualsiasi sistema operativo**.

Gli **svantaggi** dei thread ULT sono:

- **la chiamata a sistema da parte di un thread blocca tutti i thread del processo**;
- **il kernel assegna un processo ad un singolo processore quindi non si può avere multiprocessing a livello di thread**, cioè thread dello stesso processo su più processori.
  Esistono delle soluzioni parziali a questi problemi che sono:
  - sviluppo dell'applicazione a livello di processi ma andremmo a perdere tutti i vantaggi dei thread;
  - **jacketing:** conversione di una chiamata bloccante in una non bloccante, ad esempio nel caso di I/O si invoca una procedura di jacketing che verifica se il dispositivo è occupato, in caso affermativo il thread passa in ready e un altro thread va in run.

2.7.2 Kernel Level Thread

Per quanto riguarda i kernel level thread il lavoro di gestione dei threads è svolto dal kernel e il **modello è uno a uno**.



A livello utente, c'è una API che consente l'accesso alla parte del kernel che gestisce i thread, i compiti del kernel sono:

- mantenere info sul contesto del processo;
- mantenere info sul contesto dei thread;
- mantenere info sullo scambio dei messaggi tra threads.

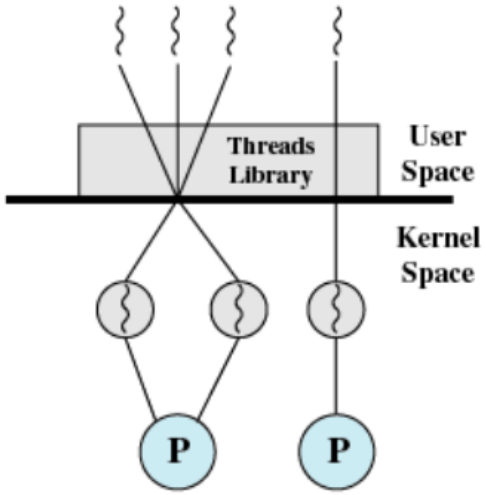
La **schedulazione** per questa tipologia di thread viene **effettuata a livello di thread**

- se un thread di un processo è bloccato, un altro thread dello stesso processo può essere eseguito;
- threads di uno stesso processo possono essere schedulati sy diversi processori.

Lo **svantaggio** dei kernel level thread è l'**overhead** perchè il trasferimento del controllo da un thread a un altro richiede l'intervento del kernel.

2.7.3 Approcci misti

Esiste un altro tipo di modello, il **modello molti a molti**, cioè più thread di livello utente sono in corrispondenza con più thread di livello kernel.



In questo approccio i thread vengono creati nello spazio utente e vari thread di uno stesso processo possono essere eseguiti contemporaneamente su più processori e una chiamata bloccante non blocca l'intero processo.
Per fare questo, però, c'è necessità di comunicazione fra kernel e libreria di thread per mantenere un appropriato numero di kernel thread allocati nell'applicazioni.
Il **Light Weight Process** o **LWP**, è una struttura intermedia che appare alla libreria dei thread utente come un processore virtuale sul quale schedulare l'esecuzione.
Ad esempio, un'applicazione CPU-bound su un sistema monoprocessore implica che solo un thread alla volta può essere eseguiti quindi sarà sufficiente un unico LWP per thread. Su una applicazione I/O bound, invece, è richiesto un LWP per ciascuna chiamata di sistema bloccante.

2.7.4 Relazione tra thread e processi

Thread:processi	Descrizione	Sistemi
1:1	Ogni thread di esecuzione è un processo unico con il proprio spazio di indirizzamento e le proprie risorse	Molte implementazioni di UNIX
M:1	Ogni processo ha associato un proprio spazio di indirizzamento e delle risorse. In ogni processo si possono creare ed eseguire molti thread	Windows NT, Solaris, MACH, OS/2, OS/390
1:M	Un thread può spostarsi da un processo all'altro, questo permette di spostare facilmente i thread tra sistemi diversi	Ra(Clouds), Emerald
M:M	Combina le proprietà degli approcci M:1 e 1:M	TRIX

Gli ultimi approcci vengono usati negli ambienti distribuiti dove i thread possono spostarsi tra più calcolatori.

2.8 Symmetric Multi Processing (SMP)

Un calcolatore con più processori presenta diverse caratteristiche:

- i processori condividono le stesse risorse;
- tutti i processori possono effettuare le stesse funzioni;
- ogni processore esegue una stessa copia del SO;
- ogni processore gestisce la schedulazione dei processi o thread disponibili.

Non è facile, però, eseguire i compiti sopra elencati in quanto:

- i processori non devono schedulare lo stesso processo;

- ci deve essere comunicazione tra processori per quanto riguarda la memoria condivisa in quanto c'è la possibilità di effettuare accessi simultanei alla memoria;
- ci deve essere coerenza della cache che viene risolto a livello hardware.

Il sistema multiprocessore **deve essere trasparente all'utente** in quanto il programmatore deve operare cose se fosse in multiprogrammazione su monoprocessore.
Nella progettazione di un Sistema Operativo per i sistemi multiprocessore ci sono diversi punti critici quali:

- processi e thread del kernel concorrenti, cioè l'esecuzione su diversi processori non deve compromettere le strutture di gestione del Sistema Operativo;
- la schedulazione, non devono esserci conflitti;
- la sincronizzazione: la mutua esclusione e l'ordinamento degli eventi;
- gestione della memoria condivisa;
- tolleranza ai guasti: in caso di perdita di un processore devono essere aggiornate le strutture di controllo del Sistema Operativo.

2.9 Stati dei thread di Windows

Gli stati dei thread di Windows sono:

- **Ready**;
- **Standby**: legato alla disponibilità del processore (SMP) richiesto per il thread. Se la priorità è sufficientemente alta il processo in running può essere interrotto;
- **Running**;
- **Waiting**: in attesa di evento I/O, attesa per sincronizzazione;
- **Transition**: thread pronto per l'esecuzione ma le risorse non sono disponibili;

- **Terminated**

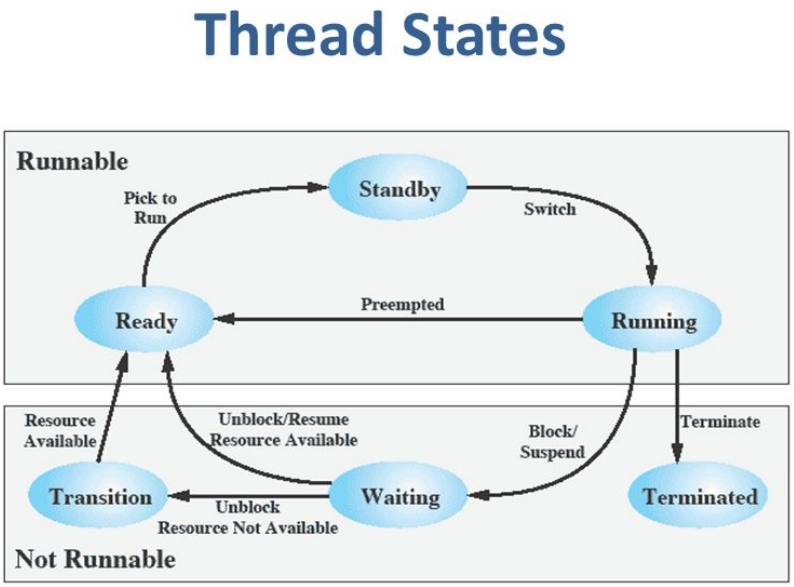
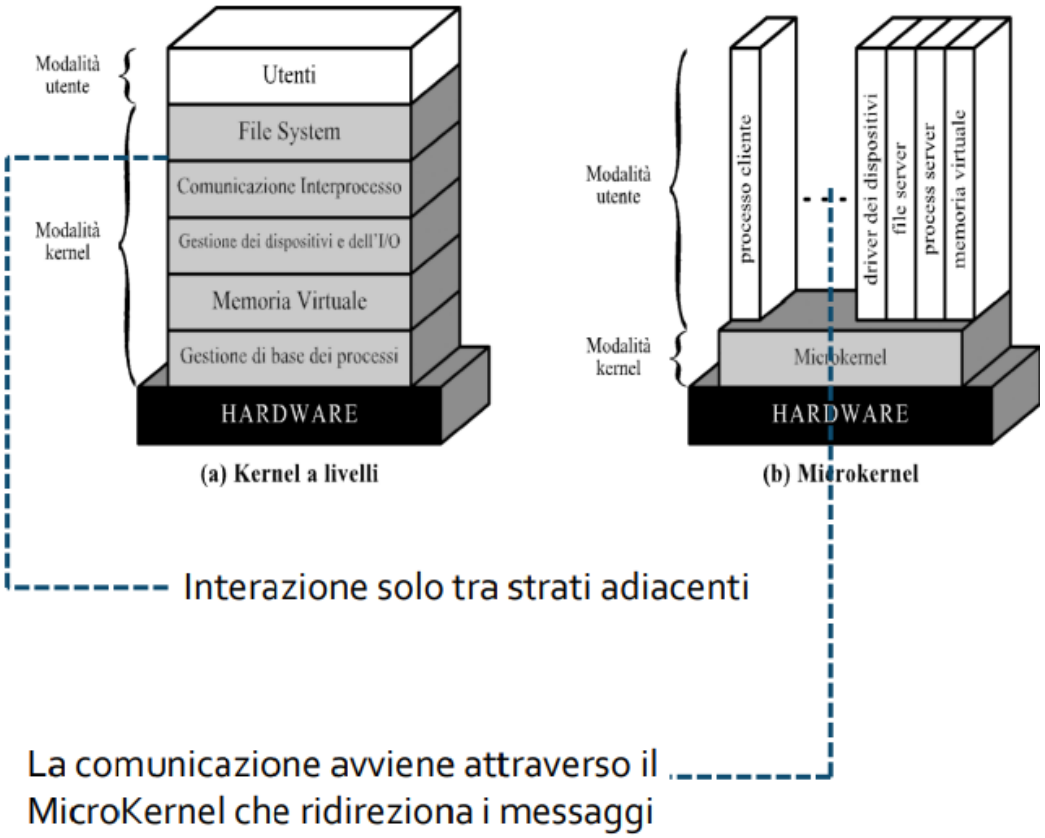


Figure 4.12 Windows Thread States

25

C'è il supporto del SMP quindi tutte le tipologie di thread possono essere eseguiti su ogni processore; il primo thread in ready viene assegnato al primo processore disponibile; thread appartenenti allo stesso processo possono essere eseguiti contemporaneamente su diversi processori; esecuzione di un thread sempre sullo stesso processore cosi i dati sono ancora nella cache del processore.

2.10 MicroKernel



Il microkernel è un piccolo nucleo del sistema operativo che ne contiene solo le funzioni essenziali, mentre i servizi inclusi tradizionalmente nel Sistema operativo, come mostrato nella figura, sono sottosistemi esterni al microkernel ed eseguiti in modalità utente, come la gestione della memoria virtuale, i driver dei dispositivi, il file system ecc.

2.10.1 Vantaggi del microkernel

Il microkernel presenta diversi vantaggi quali:

- **interfaccia uniforme**: i moduli usano le stesse interfacce per le richieste al microkernel;

- **estensibilità**: introduzione di nuovi servizi o modifiche non richiedono modifiche del microkernel;
- **flessibilità**: a seconda delle applicazioni certe caratteristiche possono ridotte o potenziate per soddisfare al meglio le richieste dei clienti, come Windows home o professional o ultimate;
- **portabilità**: il cambio dell'hardware cambierà unicamente la modifica del microkernel;
- **affidabilità**: lo sviluppo di piccole porzioni di codici ne permette una migliore ottimizzazione e test;
- **supporto ai sistemi distribuiti**: ogni servizio è identificato da un numero nel microkernel e una richiesta da client non è necessario che sappia dove si trova il server in grado di soddisfare la stessa perchè la messaggistica è gestita dal microkernel.

2.10.2 Design del microkernel

Il microkernel deve contenere:

- le funzioni che dipendono direttamente dall'hardware come la gestione degli interrupt e I/O;
- le funzioni per la comunicazione tra processi;
- gestione primitiva della memoria.

L'utilizzo del microkernel, però, porta diverse problematiche riguardanti le prestazioni del sistema perchè costruire, inviare, accettare e decodificare un messaggio costa più che fare una chiamata al SO e ci sono due possibili soluzioni a questo problema: la prima è quella di aggiungere funzionalità al microkernel, come fa il MACH OS in modo da ridurre il numero di cambiamenti di stato das utente a kernel però questo porta a interfacce minime alla riduzione di flessibilità ecc.; la seconda è un'ulteriore riduzione del microkernel.

2.10.3 Funzioni minime del microkernel

La prima funzione principale o minima del microkernel è la **gestione primitiva della memoria**. In questa funzione troviamo un modulo esterno al microkernel che mappa pagine virutali in pagine fisiche e il mapping viene conservato in memoria principale.

- un'applicazione che accede ad una pagina che non si trova in memoria genera un page fault;
- l'esecuzione passa al microkernel che invia un messaggio al paginatore comunicando la pagina richiesta;
- la pagina viene caricata e in questa fase il paginatore e il kernel collaborano per il mapping della memoria reale in memoria virtuale;
- quando viene caricata la pagina il pager invia un messaggio all'applicazione.

La seconda funzione del microkernel è la **comunicazione tra processi**. Un messaggio fra processi è formato dall'intestazione, dal corpo dal puntatore e le informazioni di controllo:

- **Intestazione**: mittente e ricevente;
- **Corpo**: dati del messaggio;
- **Puntatore**: informazioni di controllo del processo e blocco dati.

Ad ogni processo viene associata una **porta**: una capability list indica chi può inviare messaggi e questa porta viene amministrata dal kernel.
La terza funzione del microkernel è la **gestione degli interrupt e I/O**. Il microkernel non gestisce direttamente gli interrupt ma li riconosce e **trasforma l'interrupt in messaggio a livello utente**, che invia al processo che gestisce gli interrupt.

```
1     driver thread:
2     do
3       wait(msg, mittente);
4       if (mittente=mio_intterupt_hardware)
5         then leggi/scrivi le porte di I/O;
6       azzera l'interrupt hardware
7       else...
```

```
8         Endif
9       Enddo
10
```

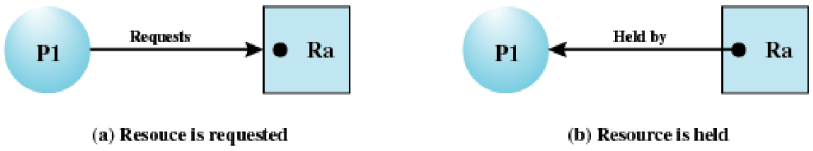
Capitolo 3

Gestione delle risorse e problematiche connesse

I processi utilizzano delle risorse che sono gestite e amministrate dal SO e il **sistema operativo decide in merito all'assegnazione delle risorse ai processi**, disponendo l'ordine di accesso dei processi a queste e le modalità di accesso.
I processi utilizzano le risorse per tramite del Sistema Operativo, che trasmette le operazione agli strati sottostati per conto dei processi.
La gestione delle risorse può portare a diverse problematiche che devono essere accuratamente attenzionate e queste problematiche sono legate alla concorrenza e alla mutua esclusione delle risorse.
La**mutua esclusione** delle risorse significa che **una risorsa singola può essere assegnata a un solo processo alla volta**.

3.1 Grafo di allocazione delle risorse

Un grafo di allocazione delle risorse rappresenta in che modo le risorse sono assegnate ai processi e le richieste dei processi:



(a) Resouce is requested (b) Resource is held

3.2 Stallo dei processi

Lo **stallo dei processi** o **deadlock** è una delle condizioni problematiche che possono verificarsi nell'ambito della gestione delle risorse.
La definizione di deadlock è: **un insieme di processi è detto in stallo se ciascun processo nell'insieme è in attesa di un evento che solo un altro processo dello stesso insieme può generare**.
Lo stallo comporta che, essendo tutti i processi in attesa di un evento, sono nello stato di blocked, questa situazione comporta che nessuno dei processi in stallo può:

- passare in esecuzione perchè necessitano del verificarsi di un evento;
- rilasciare le risorse in quanto non può passare in esecuzione

I processi in stallo sono congelati in questa situazione a meno di un intervento da parte del SO o dell'amministratore.

3.2.1 Condizioni per lo stallo

Lo stallo dei processi per verificarsi necessita della congiunzione di quattro condizioni:

1. **mutua esclusione**, un solo processo alla volta può utilizzare una risorsa;
2. **hold & wait**, un processo può mantenere il possesso delle risorse allocate mentre attende di averne altre;
3. **assenza di prerilascio**, i processi non possono essere forzati a rilasciare in anticipo le risorse acquisite;
4. **attesa circolare** un processo aspetta una risorsa che è occupata da un altro processo in attesa circolare

Le prime tre condizioni sono delle condizioni derivanti direttamente dalla progettazione e sono necessarie ma non sufficienti, la quarta è la condizione che fa verificare la condizione di stallo e che rende sufficienti tutte le condizioni

3.2.2 Strategie per affrontare lo stallo

Sono possibili quattro diverse strategie per affrontare il problema dello stallo dei processi:

1. **ignorare il problema**, l'algoritmo dello struzzo;
2. **consentire il verificarsi dello stallo, rilevarlo e risolverlo**
3. **evitarlo con politiche di allocazione**: le tre condizioni necessarie sono permesse, un algoritmo verifica dinamicamente che una richiesta non produca una situazione di stallo;
4. **impedirlo rimuovendone le condizioni**: progettare un SO in modo che la possibilità di avere uno stallo sia esclusa a priori tramite la negazione di una delle 4 condizioni.

3.3 Algoritmo dello struzzo

L'algoritmo dello struzzo pretende che il problema non esista ed è una soluzione ragionevole se:

- lo stallo capita di rado;
- il costo per evitare lo stallo è troppo elevato.

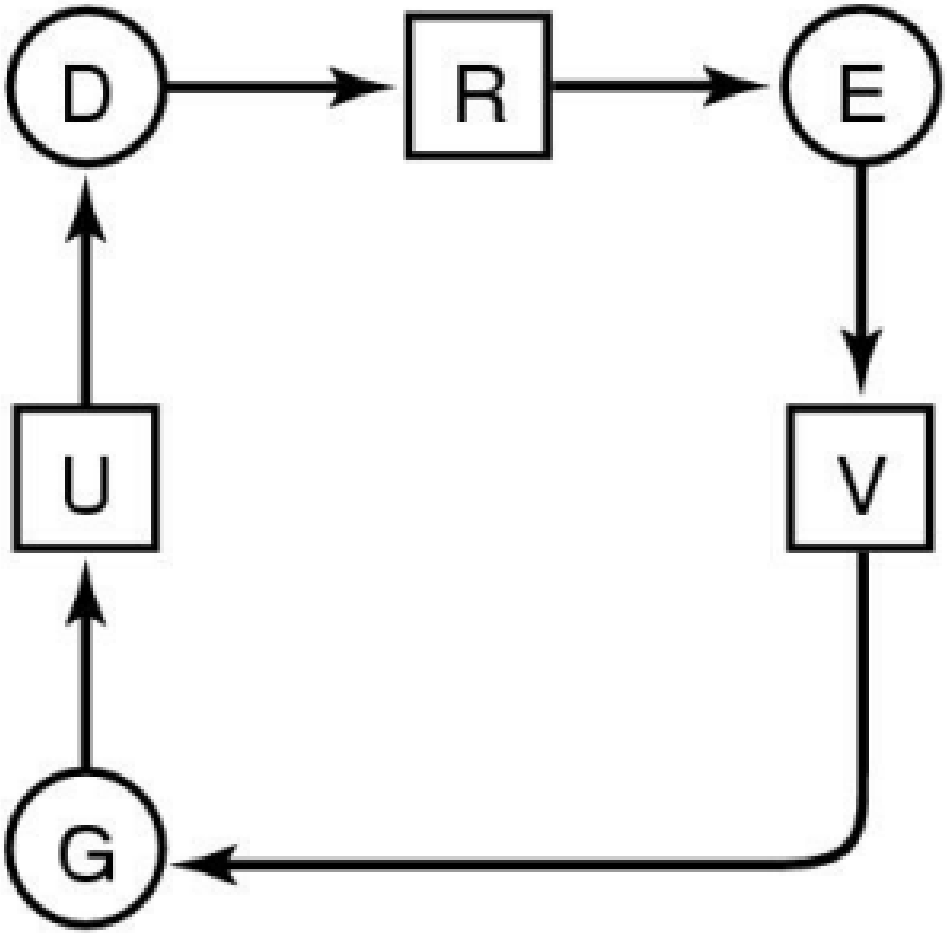
Si ha quindi un compromesso tra convenienza e correttezza.
Molte volte si utilizza questo approccio infatti Windows e UNIX utilizzano questo approccio.

3.4 Consentire lo stallo, rilevarlo e risolverlo

La seconda strategia possibile è quella di **consentire il verificarsi dello stallo, rilevarlo e risolverlo**.
Questa strategia prevede di operare normalmente e verificare periodicamente se si è verificato uno stallo.
Questa strategia può essere divisa in due fasi:

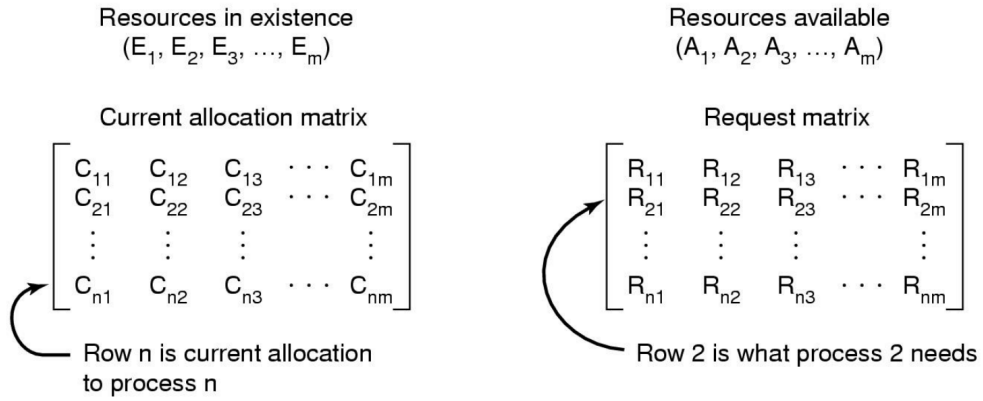
1. rilevare lo stallo;
2. eliminare lo stallo.

3.4.1 Rilevare lo stallo



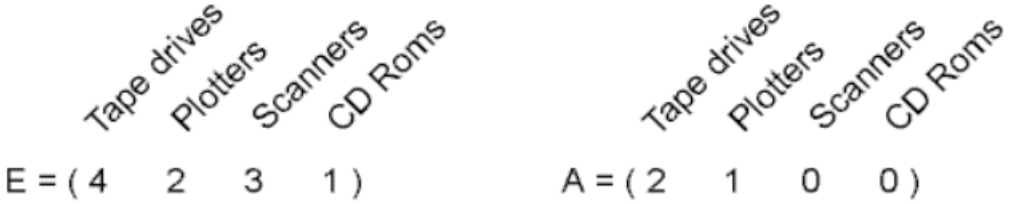
Avendo a disposizione una sola risorsa di ogni tipo, cioè una sola stampante, un solo mouse ecc, si può rilevare lo stallo costruendo il grafo di assegnazione delle risorse e un ciclo all'interno del grafo denota uno stallo.
Questo è irragionevole da implementare in quanto molto complesso perchè le risorse disponibili sono molte ed è applicabile solo nei casi di una risorsa per tipologia.
Perciò, serve una soluzione che funzioni in generale, cioè nei casi in cui si

hanno più risorse per tipologia.
Definiamo le strutture dati di cui necessitiamo:



Notazioni:

- Sia R_i l'i-esima riga della matrice R
- Dati due vettori $X = [x_1, \ldots, x_m]$ e $Y = [y_1, \ldots, y_m]$
  $X \leq Y \iff \forall\, i \in [1, m]\ x_i \leq y_i$





L'**algoritmo di individuazione** dello stallo funziona nel seguente modo:

1. inizialmente ogni processo P_i non è marcato
2. Per ogni processi non marcato P_i tale che $R_i \leq A$, cioè individuo i processi che possono ottenere le risorse necessarie alla sua terminazione
   - $A = A + C_i$
   - marca P_i Qui il processo può terminare la sua normale esecuzione e ne rilascia le risorse
3. Se rimangono processi non marcati i processi non marcati sono in stallo, cioè nessuno di loro può acquisire risorse sufficienti a terminare, tenendo occupate le risorse già prese.

Quando e quanto spesso invocare l'algoritmo dipende da:

- frequenza presunta con la quale si verifica lo stallo;
- numero di processi coinvolti nello stallo.

Uno stallo si verifica quando un processo avanza una richiesta che non può essere soddisfatta immediatamente.
Utilizzo dell'algoritmo a ogni richiesta: consente la determinazione dello stallo e del processo la cui richiesta ha dato origine allo stallo, però questo appesantisce di molto il sistema.
Utilizzare l'algoritmo quando la percentuale di utilizzo delle risorse scende al di sotto di una soglia
Utilizzare l'algoritmo ad instanti arbitrari: nel grafo di assegnazione delle risorse potrebbero esistere molti cicli e diventa difficile determinare quale processo ha causato lo stallo.

3.4.2 Eliminare lo stallo

Si può operare secondo diverse modalità:

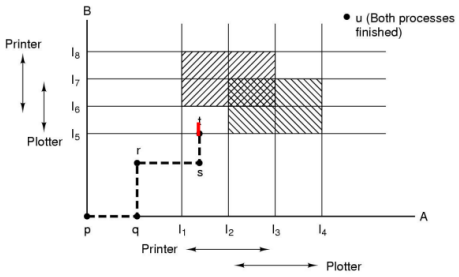
- tramite **terminazione dei processi**:
  - abort di tutti i processi coinvolti nello stallo. Soluzione comunemente adottata.

  - abort di un processo alla volta fino a quando il ciclo non è eliminato. Dopo ogni abort si deve rieseguire l'algoritmo di determinazione dello stallo. L'ordina con cui abortire i processi è dato dalla funzione di minimo costo basata su:
    * priorità dei processi;
    * tempo trascorso in esecuzione e necessario al completamento;
    * risorse già utilizzate e risorse richieste;
    * tipo di processo.

- tramite **prelazione di risorse**
  - selezionare un processo vittima tramite la funzione di minimo costo;
  - rollback del processo a cui è stata sottratta la risorsa - return ad uno stato sicuro per poterlo riavviare in seguito.
    Salvataggio periodico dello stato dei processi. In caso di stallo:
    * si ripristina il processo all'ultimo stato salvato;
    * si fa ripartire il processo, il non-determinismo dei processi dovrebbe garantire il non ri-verificarsi dello stallo.

    In generale conviene uccidere il processo
  - starvation, alcuni processi potrebbero essere selezionati costantemente come vittime e quindi bisogna includere come fattore il numero di rollback.

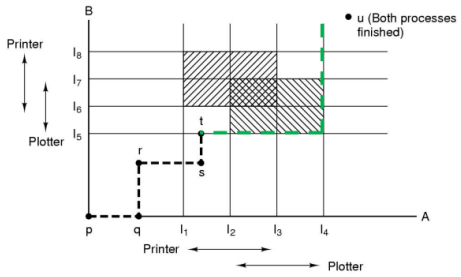
3.5 Evitare lo stallo con politiche di allocazione



Ordine di esecuzione: I_1, I_5

Si arriva allo stallo, in quanto non vi sono percorsi percorribili:

- Printer è assegnato ad A, Plotter a B
- Per proseguire, A necessita del Plotter
- Per proseguire, B necessita del Printer



Ordine di esecuzione: $I_1, I_2, I_3, I_4, I_5, I_6, I_7, I_8$

Non si verifica stallo

Questa strategia presenta alcuni vantaggi e svantaggi:

- **Vantaggi**: non è necessario interrompere i processi e riportarli in uno stato precedente
- **Svantaggi**:
  - il numero massimo di risorse necessitate da ogni processo deve essere noto a priori;
  - i processi devono essere indipendenti: l'ordine di esecuzione non deve essere vincolato da esigenze di sincronizzazione;
  - deve esserci un numero fissato di risorse da allocare;
  - quando un processo richiedere risorse potrebbe essere posto in attesa;
  - quando un processo ottiene tutte le risorse di cui necessita, deve rilasciarle in un tempo finito.

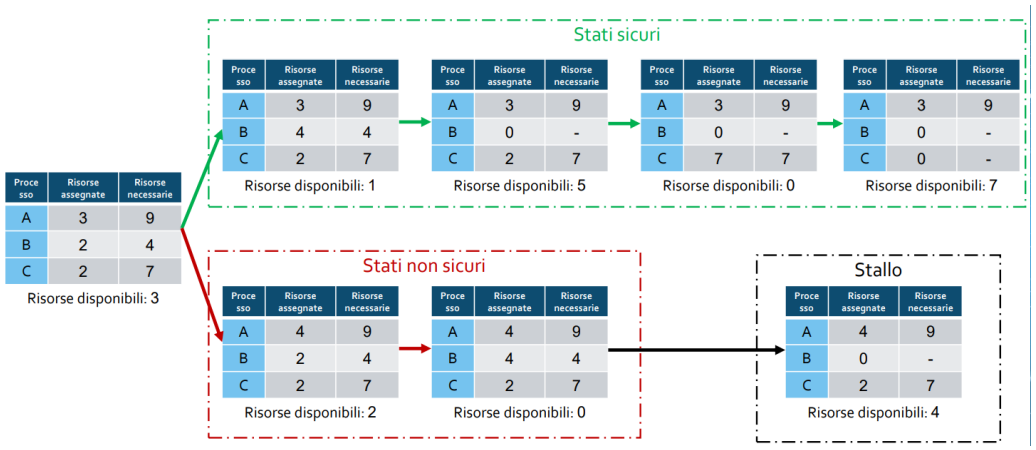
3.5.1 Stato sicuro e insicuro

Si possono definire due stati del sistema rispetto alle risorse assegnate:

1. **stato sicuro**: esiste almeno una sequenza di esecuzione dei processi che ne consente la terminazione senza incorrere in una situazione di stallo

2. **non sicuro**

Lo stato sicuro garantisce l'impossibilità di verificarsi dello stallo, mentre, lo stato non sicuro indica che c'è la possibilità ma non la certezza che si verifichi uno stallo.
Deadlock avoidance significa quindi assicurarsi che il sistema non entri mai in uno stato non sicuro.
Segue un'esempio di stato sicuro e insicuro:



3.5.2 Algoritmo del banchiere

La strategia di evitare lo stallo secondo politiche di allocazione vede la sua applicazione con l'uso dell'**algoritmo del banchiere**.
Questo algoritmo assegna risorse al processo solo se il sistema resta in uno stato sicuro, cioè i processi procedono, terminano e restituiscono le risorse al sistema e lo stato sicuro viene determinato mediante l'uso dell'algoritmo di rilevazione dello stallo.
Codice dell'algoritmo:

```
1     //global data structures
2     struct state
3     {
```

```
    int resource[m];
    int available[m];
    int claim[n][m];
    int alloc[n][m];
  }

  //resource alloc algorithm
  if (alloc [i, *] + request [*] > claim [i, *])
    < error >;
  else if (request [*] > available [*])
    < suspend process > ;
  else
  {
    < define newstate by:
    alloc [i, *] = alloc [i, *] + request [*];
    available [*] = available [*] - request [*] >
  }
  if (safe (newstate))
    < carry out allocation >;
  else
  {
    < restore original state >;
    < suspend process >;
  }

  //test for safety algorithm (banker's algorithm)
  boolean safe (state S)
  {
    int currentavail[m];
    process rest[< number of processes >];
    currentavail = available;
    rest = (all processes);
    possible = true;
    while (possible)
    {
      < find a process $P_k$ in rest such that
        claim [k, *] - alloc [k, *] <= currentavail;>
      if (found)
      {
        currentavail = currentavail + alloc[k, *];
        rest = rest - {$P_k$};
      }
      else
        possible = false;
    }
```

```
49
50          return (rest == null);
51      }
52
```

3.6 Impedire lo stallo rimuovendone le condizioni

Questa strategia impedisce il verificarsi dello stallo rimuovendo una delle quattro condizioni e quindi, si impongono dei vincoli sulla richiesta delle risorse.
Le classi di prevenzione sono:

- **metodi indiretti**: prevengono il verificarsi di una delle tre condizioni necessarie;
- **metodi diretti**: prevengono il verificarsi dell'attesa circolare

3.6.1 Negare la mutua esclusione

- un processo non deve mai attendere una risorsa condivisibile;
- devono essere possibili accessi multipli contemporanei alle risorse condivise;

Questo algoritmo potrebbe funzionare in alcuni casi, ma in altri casi, ad esempio quando la risorsa in questione è il mouse, non possono esserci accessi multipli e quindi l'algoritmo non è applicabile in generale.
Come dicevo, in alcuni casi specifici, ad esempio nel caso in cui la risorsa in questione è una stampante, che è un dispositivo gestibile con spool, l'algoritmo è utilizzabile:

- i processi scrivono l'output in un'area di spool;
- solo il gestore della stampante richiede e usa la stampante;

Non tutti i dispositivi però possono essere gestiti con spool, inoltre, si può verificare lo stallo sullo spool.
Il principio è quello di non assegnare una risorsa se non strettamente necessario e fare in modo che il minor numero di processi possa richiedere una risorsa.

3.6.2 Negare l'hold & wait

Con questa strategia ogni processo dovrebbe richiedere all'inizio dell'esecuzione tutte le risorse. Questo però comporta dei problemi perchè il processo potrebbe non sapere di quali risorse avrà bisogno, il processo entra in blocked fino a quando tutte le richieste non vengono soddisfatte contemporaneamente e potrebbe bloccare delle risorse che altri processi potrebbero utilizzare.
Questo quindi è un approccio inefficiente perchè prima di andare in run tutte le risorse dovrebbero essere disponibili quando in realtà potrebbe procedere solo con una parte di esse e le risorse che vengono assegnate a un processo potrebbero rimanere inutilizzate per molto tempo e quindi gli altri processi rimangono in attesa per un tempo non definito.

3.6.3 Negare l'assenza di prerilascio

Questo approccio può essere applicato in 2 modi:

1. un processo che non riesce ad ottenere le risorse di cui necessita, rilascia quelle che detiene per richiederle nuovamente in un istante successivo;

2. il Sistema Operativo può richiedere il pre-rilascio delle risorse al processo che le detiene per assegnarle al richiedente

Questo approccio è applicabile se lo stato della risorsa si può salvare facilmente, come nel caso della CPU, ma non applicabile nel caso delle stampanti ad esempio.
Infatti, ipotizziamo un caso reale di utilizzo della stampante: supponiamo che una stampante stia stampando dopo una richiesta da parte del processo e successivamente questo processo viene forzato a rilasciare le risorse per assegnarle a un altro processo e la stampante smette di stampare quello che stava stampando precedentemente per iniziare a stampare ciò che richieder il nuovo processo e questo è assurdo.

3.6.4 Negare l'assenza circolare

In questo approccio si definisce un ordine lineare di numerazione per tutte le risorse, quindi se un processo richiedere una risorsa R, successivamente potrà richiedere solo una risorsa che nell'ordinamento segue R.

Ad esempio, se R_i precedere R_j il processo potrà richiedere le risorse nell'ordine R_i, R_j.
Questo metodo però può essere inefficiente.

3.7 Concorrenza

Dobbiamo definire alcuni concetti:

- **multiprogrammazione**: gestione di più processi su un singolo processore;
- **multiprocessing**: gestione di più processi su più processori;
- **processi distributi**: cluster.

La concorrenza è la **competizione tra processi (threads) per ottenere e condividere le risorse**

3.7.1 Terminologia della concorrenza

- **sessione critica:** porzione di codice all'interno di un processo (thread) che richiede accesso a risorse condivise;
- **deadlock**: situazione nella quale due o più processi sono impossibilitati dal procedere perchè sono in attesa l'uno dell'altro;
- **livelock**: situazione nella quale due o più processi cambiano continuamente il proprio stato a causa del cambiamento di stato degli altri senza fare del lavoro utile;
- **mutua esclusione**: requisito per il quale, quando un processo è nella propria sezione critica, nessun altro processo può essere nella propria sezione critica se questa fa riferimento a risorse condivise con il primo processo;
- **race condition**: situazione nella quale thread o processi leggono e scrivono un dato condiviso e il risultato dipende dalla loro velocità reciproca;
- **starvation**: situazione nella quale un processo non riceve mai l'utilizzo di una risorsa e viene costantemente scavalcato da altri processi.

3.7.2 Vantaggi e problemi della concorrenza

I vantaggi della concorrenza riguardano un migliore utilizzo delle risorse e i benefici che ne si ricavano sull'esecuzione nonostante il sovraccarico a causa del context switching.
I problemi della concorrenza, nel caso di singolo processore, possono essere:

- una **condivisione pericolosa**: ordine delle operazioni di lettura e scrittura su aree di memoria condivise;
- difficoltà nell'assegnare le risorse ai processi in maniera ottimale;
- difficoltà nella rilevazione degli errori nel codice e dei conflitti di interlacciamento, cioè l'esecuzione dei processi paralleli.

Consideriamo un esempio della procedura *echo*:

```
1    char in, out;
2    void echo()
3    {
4      scanf("%c", &in);
5      out = in;
6      printf("%c", out);
7    }
8
```

Supponiamo che si siano due processi concorrenti, P_1 e P_2:

Processo P1 – Processore1	**Processo P2 – Processore2**
.....	.....
scanf("%c", &in);	.....
.....	scanf("%c", &in);
out = in;	out = in;
printf("&c", out);	.....
.....	printf("&c", out);

In questo caso il carattere letto da P_1 viene perso perchè P_1 esegue la scanf, viene interrotto, P_2 viene eseguito per intero e P_1 riprende l'esecuzione ma ora il carattere letto da P_1 viene sovrascritto da P_2.
La soluzione di questo problema è la **mutua esclusione**, cioè l'esecuzione seriale dei programmi.

3.7.3 Mutua esclusione

I meccanismo che provvedono alla mutua esclusione devono garantire a:

- un solo processo alla volta deve accedere alla sezione critica;
- un processo fuori dalla sezione critica non deve interferire con il processo nella sezione critica;
- ogni processo deve poter accedere dopo un tempo finito di attesa in coda alla sezione critica, quindi non c'è starvation o stallo;
- se nessun processo è nella sezione critica, un processo deve avere la possibilità di entrarci senza attese;
- non ci devono essere supposizioni sulla velocità di esecuzione relative dei processi;
- il tempo di permanenza nella sezione critica è definito.

I meccanismi che provvedono alla mutua esclusione sono:

- **Approcci software**: i processi, senza aiuto del SO o del linguaggio di programmazione, devono coordinarsi tra loro tramite l'**algoritmo Dekker**
  - aumento del tempo di esecuzione;
  - errori frequenti
- utilizzo di particolari **istruzioni macchina** come *test-set, scambio*:
  - riduzione del sovraccarico;
  - non soddisfacenti.
- **Supporto del sistema operativo o del linguaggio di programmazione**:
  - scambio messaggi (Inter-Process Communication);
  - semafori;
  - monitor.

3.7.4 Algoritmo di Dekker

Tentativo n.1 Nel primo tentativo dell'algoritmo di Dekker si applica, quello che viene chiamato **protocollo dell'Igloo**, che fa riferimento ai rifugi a forma di cupola costruiti con blocchi di neve la cui porta permette l'entrata di una sola persona alla volta. Applicando questo concetto ai processi, essi, prima di entrare nella sezione critica, controllano uno alla volta una variabile *turno*.
Algoritmo:

```
1     var turno = 0..1; //variabile globale condivisa con due
      valori, 0 e 1
2     Processo 0:
3     while (turno != 0)
4         {nulla} //busy wait
5     <sezione critica>;
6     turno = 1;
7
8     Processo 1:
9     while (turno != 1)
10        {nulla} //busy wait
11    <sezione critica>;
12    turno = 0;
13
```

Notiamo, quindi, che se un processo deve aspettare il suo turno entra in un ciclo dove non succede nulla e quindi viene generata la **busy wait**, cioè l'**attesa attiva** che toglie tempo utile all'esecuzione. Il pro di questo tentativo è che garantisce la mutua esclusione in quanto un processo per poter iniziare la sua esecuzione deve attendere il suo turno, d'altra parte, però, se un processo è lento questo determina la velocità di avanzamento di entrambi i processi e se un processo fallisce nella propria sezione critica e non, questo non arriverà mai all'istruzione per cambiare il turno e quindi anche l'altro rimane bloccato per sempre.

Tentativo n.2 In questo tentativo, ogni processo ha un flag relativo all'utilizzo della risorsa e può leggere il flag dell'altro senza modificarlo.
Algoritmo:

```
1     boolean flag[2];
2     Processo 0:
```

```
    //mentre il processo 1 sta utilizzando la risorsa
    //il processo 0 non fa nulla e aspetta
    while(flag[1])
      {nulla};
    flag[0] = true; //indica che il processo 0 sta
   utilizzando la risorsa
    <sezione critica>;
    flag[0] = false; //indica che il processo 0 non sta
   utilizzando la risorsa

```

L'aspetto positivo di questo approccio è che se un processo fallisce fuori dalla sua sezione critica, l'altro può continuare a lavorare, d'altra parte se un processo fallisce entro la sezione critica o prima che questo abbia settato il suo flag a false allora l'altro rimane bloccato per sempre. Inoltre, se entrambi i processi vedono il flag dell'altro a false, mettono il loro flag a true ed entrano entrambi contemporaneamente nella sezione critica andando a violare la mutua esclusione.

Tentativo n.3 In questo tentativo, viene riutilizzata la stessa tecnica del tentativo precedente con la sola differenza che i processi, prima ancora di controllare se l'altro processo è nella sezione critica, dichiarano la propria volontà di entrare in sezione critica settando la flag a true, però in questo modo, se entrambi i processi settano la propria flag a true prima ancora che uno dei due verifichi la condizione del while entrambi andranno in busy wait e si genera uno stallo. Algoritmo:

```
    boolean flag[2];

    Processo 0:
    //prima del test esprimo la volonta' di entrare in
   sezione critica
    flag[0] = true
    while(flag[1])
      {nulla};
    <sezione critica>;
    flag[0] = false;

```

L'aspetto positivo di questo approccio è che non viola la mutua esclusione, ma oltre al problema del deadlock precedentemente descritto si genera sempre il solito problema che se un processo fallisce nella propria sezione critica l'altro rimane bloccato per sempre.

Tentativo n.4 In questo approccio viene usata la tecnica precedente ma viene evitata la situazione di deadlock, generando però una situazione di attesa attiva.

```
1     Processo 0:
2     flag[0] = true
3     while (flag[1])
4     {
5       flag[0] = false;
6       <pausa>;
7       flag[0] = true;
8     }
9     <sezione critica>;
10    flag[0] = false;
11
12    Processo 1:
13    flag[1] = true
14    while (flag[0])
15    {
16      flag[1] = false;
17      <pausa>;
18      flag[1] = true;
19    }
20    <sezione critica>;
21    flag[1] = false;
22
```

Quello che succede in questo algoritmo è che i due processi settano la propria flag a true ed entrano nel ciclo, dopodichè settano la flag a false e c'è un ciclo di attesa, dopo di esso, settano nuovamente la flag a true, essenzialmente i due processi fanno questo: P0: "Vado io", P1: "Vado io", P0: "Non vado io", P1: "Non vado io" e questo genera un attesa attiva. Non è uno stallo in quanto il processo che riesce ad uscire prima dal ciclo di attesa riesce ad entrare nella sezione critica. Il solito problema che si presenta è che se un processo fallisce nella propria sezione critica l'altro rimane bloccato per sempre, d'altro canto però questo approccio garantisce la mutua esclusione.

Soluzione corretta Ogni processo ha una propria variabile **flag** che indica la volontà di entrare nella sezione critica. Inoltre, c'è una variabile **turno** che specifica chi ha il diritto di insistere nel tentativo di entrare nella propria sezione critica.
Ad esempio, se il processo 0 vuole entrare nella sezione critica pone il suo flag a true, dopodichè controlla la flag del processo 1, se questa è falsa entra

nella sezione critica, se questa è vera controlla il turno, se il turno è 0 (il suo) controlla periodicamente la flag del processo 1, se il turno è 1 quindi del processo 1 pone la sua flag a false lasciando passare il processo 1.
Algoritmo:

```
    boolean flag[2];
    int turno;
    void main() {
      flag[0] = false;
      flag[1] = false;
      turno = 1;
      processo P0;
      processo P1;
    }

    Processo P0:
    {
      flag[0] = true
      while (flag[1]) {
        if (turno == 1) {
          flag[0] = false;
          while (turno == 1)
            {nulla}; //Attesa attiva
          flag[0] = true;
        }
      }
      <sezione critica>;
      flag[0] = false;
    }

```

L'algoritmo di dekker è un algoritmo complesso e nei casi analizzati abbiamo trattato solo i casi di due processi, ma se dovessimo immaginarlo per un numero di processi maggiore questo algoritmo diventa veramente complesso. Inoltre, rimane sempre il problema dell'attesa attiva quando un processo controlla il flag dell'altro processo se pari a true e rimane sempre il problema che se un processo fallisce nella propria sezione critica l'altro rimane bloccato per sempre.

3.7.5 Supporti hardware

Nelle macchine monoprocessore i processi si alternano in esecuzione. Per garantire la mutua esclusione è possibile evitare l'interruzione di un processo

attivando e disattivando gli interrupt, quindi tramite supporto hardware. Praticamente quello che accade è questo:

$$\langle \textit{disattiva le interruzion} \rangle$$
$$\langle \textit{sezione critica} \rangle$$
$$\langle \textit{attiva le interruzioni} \rangle$$

Così facendo la mutua esclusione viene garantita, però l'efficienza del sistema peggiora in quanto il processore non può alternare i processi liberamente e non funziona su macchine SMP, cioè una macchina a più processori.
La soluzione per le macchine SMP (Symmetric Multi Processing) è di avere istruzioni macchina speciali per l'accesso a locazioni di memoria in modo atomico, cioè non interrompibile e l'accesso sequenziale ad una locazione di memoria è garantito dall'hardware. Quindi, a differenza dell'algoritmo di Dekker che era un approccio software, indipendente dal sistema operativo e dall'hardware, questa tipologia di soluzione dipende invece dall'hardware.
Le istruzioni macchina speciali che permettono l'accesso serializzato alle locazioni di memoria sono:

- test-and-set;
- scambio(swap).

Se si eseguono due test-and-set contemporaneamente queste vengono serializzate.

Test & set :

```
1     boolean test_and_set(boolean *target) {
2       boolean val = *target;
3       *target = TRUE;
4       return val;
5     }
6
```

Vediamo un utilizzo di questa funzione per garantire mutua esclusione.
Sia *lock* una variabile condivisa inizializzata a false per indicare che la risorsa è libera.

```
1     boolean lock = false;
2     while (true) {
3       while (test_and_set(&lock))
```

```
4         ; //non fare niente
5       <sezione critica>;
6       lock = FALSE; //rilascio della risorsa
7     }
8
```

La funzione *test_and_set* si salva il valore iniziale della risorsa, cioè se è accessibile o meno, lo setta a true, ma restituisce il valore iniziale. In questo modo se il processo 0 vuole utilizzare la risorsa, la funzione restituisce false e non entra nel ciclo e setta la variabile lock a true ed entra nella propria sezione critica, di conseguenza se un altro processo tenta di accedere alla risorsa, la variabile lock ora vale true perchè il processo 0 è ancora nella sezione critica, quindi la funzione restituisce true e il processo 1 entra in un ciclo di attesa fino a quando il processo 0 non esce dalla sezione critica e rilascia la risorsa.

Swap :

```
1     void swap(boolean *a, boolean *b) {
2       boolean temp = *a;
3       *a = *b;
4       *b = temp;
5     }
6
```

Vediamo un utilizzo della funzione swap per garantire la mutua esclusione. Sia *lock* una variabile condivisa settata a false per indicare che la risorsa è accessibile e *key* la volontà di entrare nella sezione critica.

```
1     while (true) {
2       key = true;
3       while (key == true)
4         swap (&lock, &key); //lock diventa true e key diventa
      false
5       <sezione critica>;
6       lock = false;
7     }
8
```

La mutua esclusione è garantita dalla stessa situazione spiegata precedentemente con la sola differenza che quando lock è true, key rimane true e quindi il secondo processo rimane in attesa fino a quando il primo processo non rilascia la risorsa.

Vantaggi e svantaggi dei supporti hardware La tecnica dei supporti hardware si può applicare a un qualsiasi numero di processi anche su multiprocessori a memoria condivisa e si possono usare per gestire anche più di una sezione critica ciascuna con una propria variabile. Il problema, però, è che c'è sempre un'attesa attiva e quindi i processi consumano tempo di CPU, ci può essere starvation perchè se gli algortimi di scheduling che sono stati scelti causano starvation, di conseguenza anche lo scheduler soffrirà di starvation, infatti la scelta di quale processo dovrà andare nella sezione critica è arbitraria. Inoltre, i supporti hardware possono causare stallo, ad esempio, se il processore è singolo, il processo P1 è in sezione critica, P2 ha priorità più alta di P1 e tenta di accedere tramite test and set o swap alla risorsa che è bloccata da P1 e nella sua attesa attiva non lascierà mai il posto a P1 in quanto ha priorità più bassa e quindi si genera uno stallo.

3.7.6 Supporto del Sistema operativo e dei linguaggi di programmazione

Le soluzioni offerte dal Sistema operativo e dai linguaggi di programmazioni sono i **semafori** e i **monitor**.
Il semaforo può essere binario, cioè gestisce una sola risorsa o contatore, cioè gestisce un lotto di risorse dello stesso tipo.
Il **semaforo** è una variabile booleana o intera sulla quale sono possibili 3 operazioni:

1. inizializzazione ad un valore non negativo, nel caso del booleano viene settato a true, nel caso sia intera viene settata al numero di risorse da gestire, ad esempio se sono 5 stampanti, viene settato a 5;

2. operazione atomica **wait()**: decrementa il valore della variabile. Se il valore della variabile diventa negativo, il processo che ha eseguito la wait viene bloccato;

3. operazione atomica **signal()**: incrementa il valore della variabile. Se il valore della variabile era negativo, uno dei processi bloccati nell'operazione di wait viene sbloccato.

Funzionamento del semaforo:

1. si associa un semaforo ad ogni risorsa condivisa;

2. il processo che vuole utilizzare la risorsa effettua un'operazione di wait;
3. il processo che rilascia la risorsa effettua il signal;
4. la variabile numerica indica il numero di istanze di una risorsa condivisa;
5. se la variabile è negativa, il valore assoluto di essa rappresenta il numero di processi in attesa

Implementazione del semaforo contatore :

```
typedef struct {
  int istanze;
  struct processo *P; //lista dei processi in coda
} semaforo;

void wait(semaforo s) {
  s.istanze--;
  if (s.istanze < 0) {
    <poni processo in coda>
    <blocca questo processo: running->blocked >
  }
}

void signal(semaforo s) {
  s.instanze++;
  if (s.istanze <= 0) {
    <rimuovi processo in coda>
    <sveglia il processo: blocked->ready >
  }
}

```

Implementazione del semaforo binario :

```
typedef struct {
  boolean val;
  struct processo *P; //lista dei processi in coda
} semaforo_bin;

void wait(semaforo s) {
  if (s.val == true) {
    s.val == false;
```

```
9       } else {
10        <poni processo in coda a P>
11        <blocca questo processo: running->blocked >
12      }
13
14    }
15
16    void signal(semaforo s) {
17      if (*P == NULL) { //coda vuota
18        s.val = true;
19      } else {
20        <rimuovi un processo dalla coda>
21        <sveglia il processo: blocked->ready >
22      }
23    }
24
```

Atomicità di wait e signal Come possiamo fare affinchè wait e signal siano atomiche?

1. **implementarle in hardware o firmware**;
2. Utilizzare l'algoritmo di Dekker o Peterson ma ci sarebbe un sovraccarico di elaborazione;
3. **Utilizzo dell'istruzione atomica test&set**
4. Se il sistema è monoprocessore basta disattivare gli interrupt durante le operazioni

I semafori non presentano attesa attiva in quanto i processi in coda vengono bloccati. Se un processo fallisce nella propria sezione critica, il semaforo si occupa comunque di dare il signal della risorsa.

Deadlock e starvation La **starvation** può verificarsi se un processo non viene mai rimosso dalla coda al semaforo, ad esempio quando c'è una gestione di tipo LIFO.
Il **deadlock** si verifica quando due o più processi sono in attesa di un evento che può essere determinato solo da uno dei processi in attesa. Ad esempio, siano S e Q due semafori inizializzati a 1:

```
    Processo 0:
    wait(S);
    wait(Q);
    .
    .
    signal(S);
    signal(Q);

    Processo 1:
    wait(Q);
    wait(S);
    .
    .
    signal(Q);
    signal(S);

```

Notiamo che P0 non rilascia S se non ottiene Q e P1 non rilascia Q se non ottiene S, quindi le signal non saranno mai eseguite e si verifica lo stallo. La situazione cambia se anche P1 fa il wait di S e poi il wait di Q.

Produttore e consumatore Questa è una tipica rappresentazione di processi concorrenti:

- uno o più produttori generano dati inserendoli nel buffer;
- un consumatore preleva i dati uno alla volta.

Per il controllo di accesso al buffer non utilizziamo il costrutto if in quando andremmo a controllare di continuo il buffer generando attesa attiva.

Monitor I semafori sono primitive potenti per gestire la mutua esclusione, ma la scrittura di un programma con utilizzo dei semafori può essere tutt'altro che semplice.
I **monitor** sono dei costrutti di sincronizzazione ad alto livello ed è un modulo software le cui caratteristiche sono:

- variabili locali accessibili solo dalle procedure definite al suo interno e non da procedure esterne;
- un processo entra nel monitor chiamando le sue procedure;

- un solo processo alla volta può essere in esecuzione all'interno del monitor, gli altri sono sospesi in attesa che il monitor diventi disponibile.

Utilizzo:

- **mutua esclusione**;
- **protezione delle strutture dati condivise**.

Si verifica la sincronizzazione tramite variabili di condizione e su queste variabili si opera tramite:

- **cwait(c)**: sospende l'esecuzione del processo chiamante sulla condizione c;
- **csignal(c)**: riattiva un processo sospeso sulla condizione c. Se non c'è nessun processo in attesa su tale condizione il segnale viene perso.

Capitolo 4

Gestione della memoria centrale

4.1 Principi di base

Nessun processo può diventare attivo prima di aver ottenuto un certo quantitativo di memoria, perciò, la gestione della memoria si occupa di allocare la memoria fisica ai processi che ne fanno richiesta.
L'utilizzo globale delle risorse e tutte le altre prestazioni di un calcolatore vengono influenzate dalle prestazioni del *modulo di gestione della memoria* sia in funzione della sua efficienza nell'allocare memoria, sia per l'influenza che può avere sullo scheduler.
Un buon gestore di memoria in un ambiente multiprogrammato deve garantire la protezione dei dati, cioè i processi attivi non devono confinare nello spazio di indirizzamento di altri processi, inoltre, deve permettere allo stesso tempo la loro condivisione affinchè i processi cooperanti possano accedere ad aree comuni di memoria.

4.2 Allocazione della memoria

L'allocazione di memoria può essere di due tipi:

1. **Contigua:** la memoria viene allocata in modo tale che ciascun oggetto occupa un insieme di locazioni i cui indirizzi sono strettamente consecutivi, i possibili tipi di allocazione contigua sono: *monoallocazione, partizionamento statico, partizionamento dinamico, segmentazione*;

2. **Non contigua:** la memoria viene allocata in modo tale che un unico oggetto logico viene posto in indirizzi non consecutivi fra loro. Durante l'esecuzione di un processo viene effettuata la traduzione degli indirizzi per ristabilire la corrispondenza tra lo spazio virtuale di indirizzamento, che è contiguo, e gli indirizzi fisici.
   I possibili tipi di allocazione non contigua sono: *paginazione, memoria virtuale.*

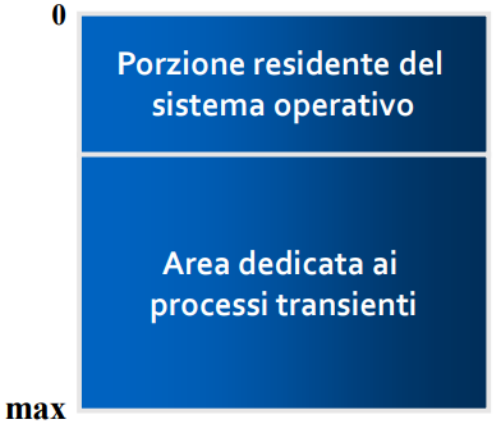
4.3 Allocazione contigua

4.3.1 Monoallocazione: Monitor Monoprocesso

La monoallocazione è lo schema più semplice per la gestione della memoria che viene suddivisa in due aree contigue:

- la prima allocata permanentemente ad una porzione residente nel sistema operativo, il Monitor;
- la seconda assegnata ai processi in esecuzione, processi utente o porzioni non residenti nel sistema operativo, per il tempo necessario al completamento.

Segue uno schema utilizzato da sistemi operativi monoprocesso:



Modalità dello schema :
Il sistema operativo effettua diversi compiti:

- tiene traccia solo della prima e dell'ultima locazione disponibile per i processi transienti;
- viene posto ad uno dei due estremi della memoria;
- ingloba i vettori di interrupt;
- altri suoi moduli temporanei vengono posti all'altra estremità della memoria.

Alla richiesta di esecuzione di un programma, il sistema operativo:

- si assicura che le dimensioni del processo siano compatibili con la memoria disponibile;
- conferisce il controllo al processo utente fino al suo completamento o ad eventuali condizioni d'errore;
- al termine del processo la memoria viene liberata e può essere assegnata ad un altro processo in attesa.

Meccanismi di protezione Raramente un monitor monoprocesso supporta meccanismi di protezione tra processi utente in quanto in ogni istante vi può essere al massimo un solo processo residente in memoria. Gli eventuali meccanismi di protezione si riferiscono alla protezione del codice del sistema operativo da eventuali sconfinamento del processo transiente in esecuzione. La protezione del sistema operativo dai processi utente è spesso effettuata tramite supporti hardware come:

- **registri barriera**: questi sono usati per tracciare un confine tra le aree dei processi di sistema e dei processi transienti. Nel registro viene salvata la prima locazione disponibile al processo in modo che il sistema operativo possa controllare un eventuale sconfinamento, per fare questo, però, ci deve essere la capacità di distinguere l'esecuzione del sistema operativo da quella dei processi utente definendo gli stati utente e supervisore;
- **diritti di accesso mediante bit di protezione**: con questo metodo ad ogni parola di memoria viene associato un bit di protezione che viene posto a 1 nelle zone che contengono il sistema operativo e a 0 nelle restanti, in questo modo i processi utente accedono solo a parole con bit 0, mentre il sistema operativo può accedere ovunque;

- **sistema operativo in memoria a sola lettura**: questo è un metodo semplice e diffuso che è scarsamente utilizzato nei personal computer in quanto poco flessibile e rende impossibile correggere o aggiornare il codice del sistema operativo.

La condivisione del codice e dei dati non ha senso negli ambienti monoprocesso e raramente viene supportata dai monitor. I programmi utente potrebbero passarsi dei dati, mediante accordi interni, ponendoli in locazioni di memoria che non vengono sovrascritte durante l'esecuzione dei processi partecipanti o mediante file.

Prestazioni :

- **Difetti:**
  - la mancanza di supporti per la multi-programmazione produce un abbassamento dell'efficienza sia della memoria che della CPU;
  - la memoria può risultare troppo grande per la maggioranza dei processi;
  - processi di dimensioni più grandi non possono essere eseguiti, oppure richiedono la tecnica dell'**overlay** cioè richiedono particolari suddivisioni del codice;
  - i programmi tendono ad essere ottimizzati rispetto alla dimensioni, comportando sacrifici di funzionalità e velocità (anche se adesso accade il contrario)
- **Pregi:**
  - basso costo di progettazione del modulo di gestione della memoria;
  - contenuti supporti hardware specifici per la gestione della memoria.

4.3.2 Partizionamento statico

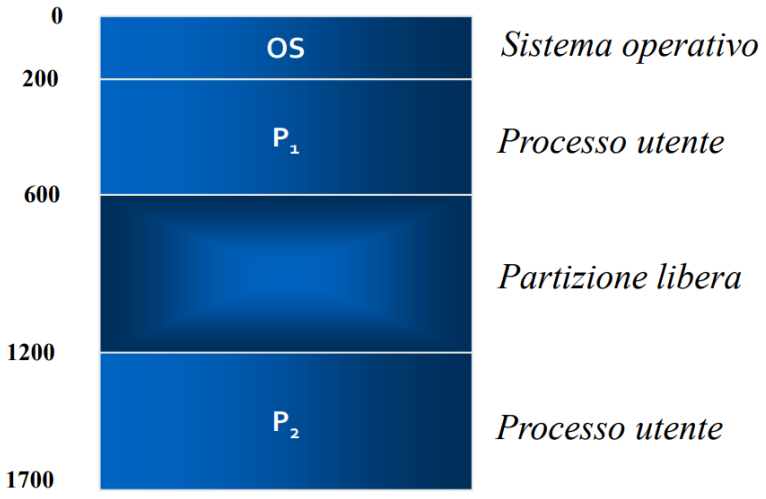
Un modo per supportare la multiprogrammazione consiste nel dividere la memoria in diverse partizioni, ciascuna delle quali può essere allocata a un processo diverso.
IL partizionamento statico implica che la suddivisione della memoria venga

fatta a priori e che da quel momento le partizioni rimangano di dimensioni fisse.
Il numero e la dimensione di ciascuna partizione vengono determinato durante la generazione del sistema considerando alcuni fattori:

- capacità della memoria fisica disponibile;
- grado desiderato di multiprogrammazione;
- dimensione dei processi più frequentemente eseguiti.

Alcuni calcolatori permettono una ridefinizione manuale delle partizioni senza dover ripetere l'intero processo di generazione del sistema.

Principi operativi :
Il sistema operativo deve tener traccia delle partizioni definite. Lo stato corrente delle partizioni e i loro attributi vengono raccolti in una struttura dati chiamata **tabella di descrizione delle partizioni o TDP**.
L'unico campo della TDP che cambia è lo stato della partizione che indica se è allocata o meno, gli altri contengono valori definiti al momento del partizionamento.
Segue un esempio di memoria partizionata:



La TDP dell'esempio precedente è:

Numero partizione	Base partizione	Dimensione partizione	Stato partizione
0	0 kB	200 kB	Allocata
1	200 kB	400 kB	Allocata
2	600 kB	600 kB	Libera
3	1200 kB	500 kB	Allocata

Quando un processo non residente deve essere creato o attivato, il sistema operativo tenta di allocare una partizione di memoria libera di dimensioni sufficienti per contenerlo, consultando la TDP. Se il test è positivo, il campo che indica lo stato della partizione viene impostata ad "Allocata" e l'immagine del processo viene caricata nella corrispondente partizione.
Quando il processo termina è necessario effettuarne uno swapping dalla memoria. Questa informazione verrà utilizzata per localizzare e aggiornare lo stato della partizione associata al valore "Libera".
I metodi più comuni per la ricerca di una partizione libera sono:

- **First fit**: consiste nell'allocare la prima partizione libera sufficientemente larga per contenere il processo;
- **Best fit:** consiste nell'allocare la più piccola delle partizioni libere che contenga il processo

Il metodo first fit è più veloce in esecuzione perchè si arresta nella ricerca alla prima partizione libera disponibile, mentre il metodo best fit deve controllare tutte le partizioni libere disponibili e poi individuare quella migliore. Tuttavia, il metodo best fit è più accurato in esecuzione e quindi può raggiungere un migliori utilizzo della memoria, rispetto al metodo first fit, riducendo al minimo la differenza di dimensione tra il processo e la partizione allocata. Però, quando il numero di partizioni è elevato nessuno dei due metodi si rivela adatto. Quindi, per riassumere, se esiste una partizione allocabile al processo richiedente:

- l'allocazione viene effettuata;
- la TDP viene aggiornata;
- nel descrittore del processo si registra la partizione associata;
- quando il processo termina la partizione viene disallocata.

Le cause di mancata disponibilità di una partizione sono:

1. **non vi sono partizioni di memoria sufficientemente grandi per contenere il processo entrante**
   Le **cause** potrebbero essere:

   - errore di scelta della dimensione delle partizioni;
   - processo eccezionalmente più grande della norma.

   La **soluzione** è di ridefinire le dimensioni delle partizioni, oppure il programma di origine deve essere progettato utilizzando uno schema di overlay;

2. **tutte le partizioni sono allocate**
   In questo caso bisogna attendere una partizione libera oppure effettuare lo swapping del processo.
   Queste soluzioni possono essere applicate anche al caso 3;

3. **alcune partizioni sono libere ma nessuna di esse è sufficientemente grande**
   In questo caso l'assegnazione delle partizioni può continuare. Vengono allocate partizioni a processi più piccoli, l'utilizzo della memoria viene mantenuto più elevato. Si può violare l'ordinamento di attivazione dei processi dallo scheduler modificando così, le prestazioni del sistema.

Il funzionamento del gestore della memoria e dello scheduler sono strettamente correlati:

- lo scheduler determina quali processi sono pronti, e quindi residenti in memoria, in attesa della CPU
- il gestore della memoria può decidere di togliere un processo in memoria per liberare una partizione.

Swapping : Lo swapping è l'operazione di rimozione della memoria di un processo sospeso ed il suo successivo caricamento. Nel partizionamento statico questo aumenta il livello di occupazione della memoria centrale e di conseguenza il livello di utilizzo della CPU.
Quando lo scheduler decide l'introduzione in memoria di un nuovo processo il gestore dello swapping ne sceglie uno da rimuovere in base a:

- dimensione della partizione richiesta;

- priorità del processo;
- tipo di evento atteso dal processo;
- tempo già trascorso in memoria.

Un processo residente in memoria, già parzialmente eseguito, è costituito da:

- codice eseguibile;
- dati finora elaborati;
- stack;
- registri di stato;
- file aperti;
- descrittore del processo.

Al momento dello swapping questi oggetti vengono registrati in un file su hard disk detto file di swapping.
Tutti i processi che hanno subito uno swapping possono essere mantenuti in un file unico per tutto il sistema o in file separati, ciascuno assegnato ad un singolo processo.
Un file di swapping viene solitamente creato al momento dell'inizializzazione del sistema e collocato su una periferica veloce di memorizzazione secondaria. L'indirizzo e la dimensione di tale file sono solitamente statici in modo da beneficiare di un indirizzamento diretto su disco.
Un parametro critico per la realizzazione dei file di swapping è la dimensione:

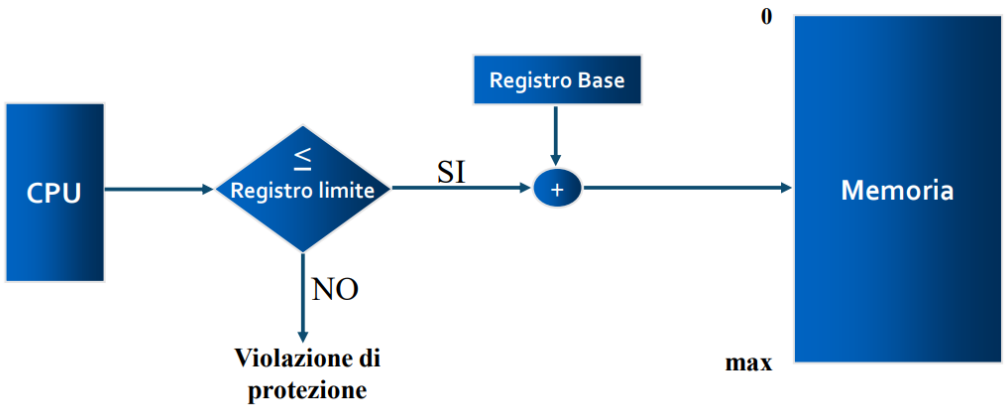
- se troppo grande spreca spazio su dischi veloci;
- se troppo piccolo può rendere indisponibile l'operazione di swapping

L'alternativa è disporre di un file di swapping per ciascun processo presente nel sistema che può essere creato dinamicamente al momento della creazione del processo o staticamente al momento della preparazione del programma. In questo modo non vige più il problema della dimensione e non c'è nessuna restrizione sul numero di processi attivi. D'altra parte, però, c'è maggior spreco di spazio e maggiori tempi di accesso ai file distribuiti sul disco.
Uno swapping efficiente richiede che i processi siano rilocabili dinamicamente:

- il processo può iniziare l'esecuzione in qualunque area di memoria;
- può essere spostato in un'altra area di memoria;
- il calcolo degli indirizzi effettuato dinamicamente ad esempio con l'uso di un *registro base*

Processi non rilocabili dinamicamente sono legati alle partizioni in cui iniziano l'esecuzioni e rendono inefficiente lo swapping. Lo swapping va utilizzato solo in casi di necessità perchè la CPU spreca un tempo elevato nelle operazioni di trasferimento.

Meccanismi di protezione L'integrità di un sistema multiprogrammato dipende anche dalla sua capacità di garantire l'isolamento tra spazi di indirizzi separati. Non solo si deve proteggere il sistema operativo dai processi utente ma bisogna anche proteggere i processi utente tra di loro perchè nessuno possa invadere lo spazio riservato all'altro.
Un sistema di protezione efficace non può prescindere dall'esistenza di un supporto hardware. Nei sistemi che utilizzano per la rilocazione i registri base, vengono usati di solito dei *registri limite* per la protezione, il cui compito è di individuare i tentativi di accesso oltre i limiti dello spazio di indirizzamento assegnato al programma in esecuzione dal sistema operativo. Il registro limite contiene il valore dell'indirizzo virtuale più alto contenuto dal programma. Ad ogni accesso alla memoria alla CPU controlla che il programma non richieda gli indirizzi oltre il limite. Le violazione vengono rilevate e riferite al sistema operativo che le gestisce. L'indirizzo più basso del processo è contenuto nel registro base:



Meccanismi di condivisione Un buon sistema di gestione della memoria deve occuparsi, oltre che della protezione, anche della condivisione controllata dei dati e codice tra processi cooperanti. I sistemi a partizioni fisse non sono adatti alla condivisione, poichè basano i propri meccanismi sull'isolamento degli spazi di indirizzamento.
Alcuni possibili metodi di condivisione sono:

1. affidare gli oggetti condivisi al sistema operativo che così è in grado di accedere a tutte le risorse. Gli svantaggi di questa tecnica è che l'area del sistema operativo dovrebbe poter variare dinamicamente e le routine dei processi devono essere linkate al sistema operativo dinamicamente;

2. ogni processo possiede una copia identica dell'oggetto condiviso, che usa e diffonde gli aggiornamenti. Il problema di questa tecnica è che se il sistema supporta lo swapping, uno o più processi potrebbero non essere in memoria e quindi non essere pronti a ricevere gli aggiornamenti.

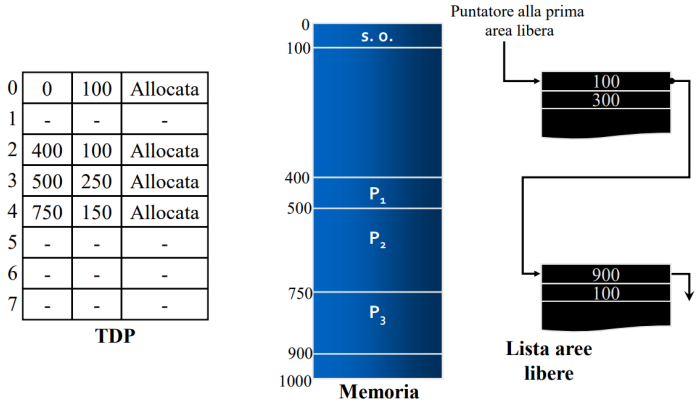
3. Collocare i dati in una partizione comune dedicata. In questo caso, però, il sistema operativo considera come violazione i tentativi dei processi di accedere a zone di memoria esterne alle rispettive partizioni. Se il sistema utilizza registri base e limite sono necessari accorgimenti per indirizzare partizioni che potrebbero essere non contigue.

Conclusioni : Il partizionamento statico è uno dei metodi più semplici di gestione della memoria, pur supportando la multiprogrammazione; richiede un hardware modesto e si adegua ad ambienti statici con carico predicibile di lavoro come ambienti dove vengono eseguiti solo applicativi, controllo di processi e sistemi di tipo bancario.
Il problema principale è la **frammentazione interna**, inoltre, può richiedere progettazione dei programmi con schemi ad overlay; non si adatta a contenere oggetti come stack che possono crescere in maniera non facilmente controllabile e il numero di partizioni fissato limita il grado di multiprogrammazione.
La **frammentazione interna** si verifica quando una partizione di memoria assegnata a un processo è più grande di quanto richiesto dal processo stesso. Questo crea dello spazio inutilizzato all'interno della partizione, che non può essere utilizzato da altri processi e quindi la memoria allocata viene in parte "sprecata" perchè non viene completamente utilizzata.

4.3.3 Partizionamento dinamico

Il sistema delle partizioni è analogo a quello del partizionamento statico. Il gestore della memoria crea ed assegna partizioni di **dimensioni variabili dinamicamente** in base alla richieste dei processi. Quando un processo termina o subisce uno swapping, il gestore della memoria restituisce lo spazio liberato all'insieme di aree di memoria libere. Il gestore della memoria può creare e allocare partizioni finchè non viene esaurita la memoria fisica o finchè non viene raggiunto il massimo grado di multiprogrammazione.

Principi operativi :
Alla richiesta di una partizione il gestore della memoria (GM) ricerca una zona di memoria libera contigua di dimensione sufficiente. Se si trova un'area adatta il sistema operativo vi ricava una partizione in modo da soddisfare esattamente le necessità del processo.
L'eventuale parte rimanente di memoria libera viene restituita all'insieme della memoria libera e posta a disposizione del modulo di allocazione. La partizione viene creata registrando la sua base, la sua dimensione e il suo stato nella TDP. Alcune di queste informazioni vengono anche registrate nel descrittore di processo. Dopo aver caricato l'immagine del processo nella partizione creata, il processo viene sottoposto al controllo di un modulo del sistema operativo che si occupa delle operazioni successive.
Quando un processo termina o subisce uno swapping il sistema operativo restituisce la partizione all'area libera e cancella la riga nella TDP. L'insieme delle aree libere varia dinamicamente, gli spazi interni all'area libera vengono utilizzati per mantenere una lista a puntatori. Ad esempio:



Gli algoritmi più comuni per selezionare un'area di memoria all'interno della quale creare una partizione sono:

- **First-fit**: questo metodo termina la ricerca quando viene individuato il primo blocco di memoria sufficientemente grande da contenere il processo;
- **Next-fit**: questo metodo è una variante del First-fit. In questo metodo il puntatore alla lista delle aree libere, dopo aver effettuato un'allocazione, viene salvato cosicchè la ricerca successiva continua da dove si era fermata la precedente;
- **Best-fit**: questo metodo alloca il blocco di memoria più piccolo che può contenere il processo;
- **Worst-fit**: questo metodo alloca il blocco di memoria più grande le cui dimensioni superino quella della partizione richiesta.

Compattazione Lo schema di partizionamento dinamico genera dei buchi di memoria inutilizzabile e questo è inevitabile. Per questo motivo può anche diventare impossibile l'allocazione di una memoria richiesta da un processo, pur contenendo la memoria globalmente un'area di dimensione sufficiente. Questo problema prende il nome di **frammentazione esterna**.
Quando la frammentazione raggiunge un grado elevato è necessario effettuare la **compattazione della memoria** che consiste nello spostare i processi residenti in memoria in modo da creare partizioni libere più grandi. Questa operazione richiede un grande costo di CPU.
La compattazione può essere effettuata:

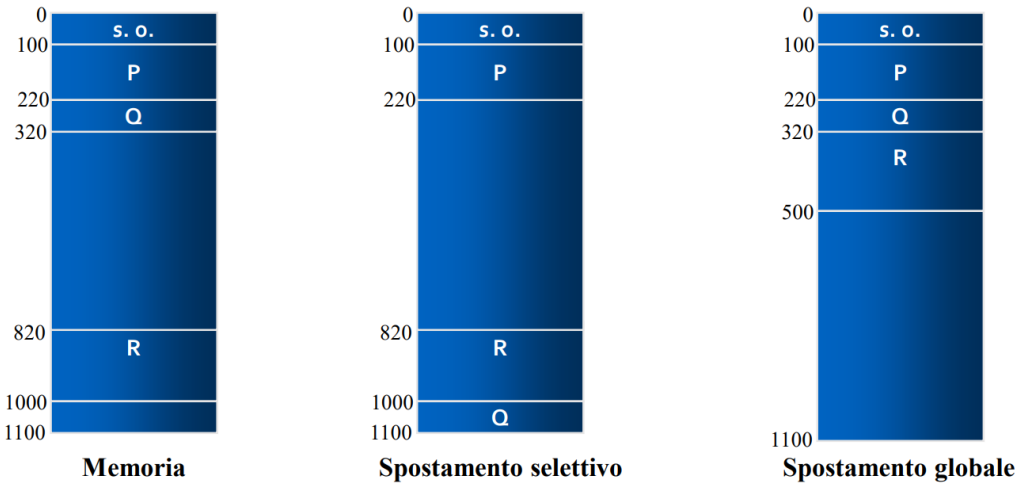
- **Continuamente:**
  - la memoria viene compattata ogni volta che un processo libera un'area;
  - la frammentazione è ridotta al minimo;
  - questa operazione ha un grande costo di CPU.
- **Su necessità:**
  - la compattazione viene effettuata quando non si riesce ad allocare memoria ad un processo richiedente;

– un controllo preliminare verifica se il totale dell'area liberabile è sufficiente a contenere il processo.

La compattazione può essere effettuata in due modi:

1. **Spostamento selettivo incrementale:**
   - ricerca di una strategia ottima di movimento per compattare al meglio la memoria;
   - consente di risparmiare tempo negli spostamenti;
   - poco utilizzata perchè richiede un sovraccarico notevole
2. **Spostamento globale:**
   - tutte le partizioni vengono rilocate ad uno dei due estremi della memoria;
   - il tempo di trasferimento delle partizioni non è ottimizzabile;
   - non richiede strategie particolari.

Segue un esempio di spostamento selettivo e globale:



Meccanismi di protezione e condivisione Le tecniche di protezione e condivisione sono analoghe a quelle del partizionamento statico, considerato che il supporto hardware è simile. Nel partizionamento dinamico è possibile una particolare forma di condivisione:

- a due partizioni contigue è consentito sovrapporsi, mettendo un'area in comune;
- questa forma di condivisione è molto restrittiva, in quanto, può avvenire solo tra due processi.

Conclusioni Il partizionamento dinamico:

- richiede un sopporto hardware modesto, analogo al partizionamento statico;
- la differenza tra i due schemi risiede sostanzialmente nel software;
- elimina la frammentazione interna ma produce quella esterna che può essere eliminata con la compattazione;
- si adegua ad ambienti con carico non predicibile di lavoro come ad esempio sviluppo di software.

4.3.4 Segmentazione

La segmentazione è uno schema di gestione della memoria che si basa sulla divisione dello spazio di indirizzamento dei processi in entità logiche che possono essere poste in aree non contigue di memoria. La segmentazione fornisce strumenti per la rilocazione dinamica, la protezione e la condivisione.
La frammentazione esterna può essere ridotta se la dimensione delle aree richieste è più piccola. Un programma eseguibile può essere diverso, ad esempio, in codice, dati e stack; ciascuno di questi oggetti può essere posto in un segmento diverso di dimensioni differenti.
La segmentazione condivide alcune proprietà con gli altri schemi di allocazione di memoria, in particolare:

- degli **schemi di allocazione contigua** relativamente ad un singolo elemento, poichè i dati di ogni singola entità logica devono essere posti in un'area contigua di memoria;

- degli **schemi di allocazione non contigua** relativamente all'intero spazio di indirizzamento del processo, poichè i blocchi logici diversi possono essere messi in segmenti non contigui.

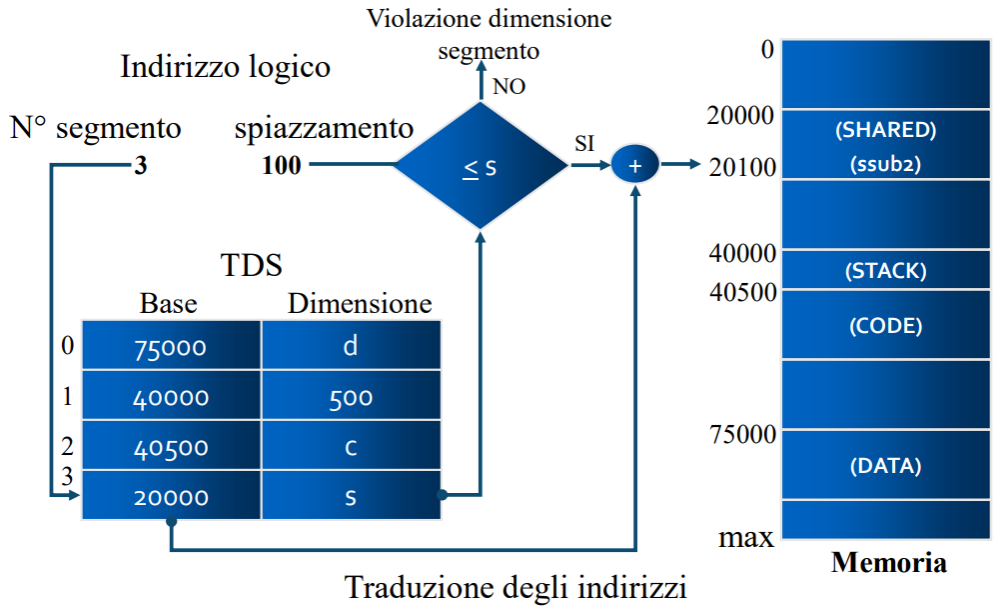
Principi operativi La raccolta degli oggetti in segmenti viene effettuata dal programmatore. Ciascun segmento inizia all'indirizzo virtuale 0. Ogni segmento ha un nome che viene poi tradotto in un indirizzo in fase di caricamento della memoria e un singolo dato all'intero del segmento viene identificato dallo spiazzamento relativo all'inizio del segmento a cui appartiene.
L'indirizzamento nella segmentazione è di tipo bidimensionale, infatti la designazione univoca di un dato o di una istruzione richiede il nome del segmento e lo spiazzamento all'interno del segmento. La memoria fisica, mantiene l'indirizzamento lineare, per cui gli indirizzi bidimensionali di segmenti virtuali devono essere tradotti in indirizzi fisici.
Il caricamento di un processo segmentato avviene nel seguente modo:

- il Sistema Operativo cerca di allocare n partizioni adatte per gli n segmenti in cui è suddiviso il processo;
- il Sistema Operativo crea un descrittore di segmento registrandovi l'indirizzo fisico in cui è stato posto il segmento e la sua dimensione;
- l'insieme dei descrittori di segmento di un processo viene raccolto nella tabella dei descrittori di segmento o **TDS**.

La conversione di un indirizzo avviene nel seguente modo:

- l'indirizzo bidimensionale virtuale, formato dal nome del segmento e dallo spiazzamento, è tradotto in modo equivalente al meccanismo del registro base;
- il segment viene utilizzato per ritrovare l'indirizzo fisico del segmento nella TDS;
- il displacement viene sommato all'indirizzo fisico del segmento per ottenere l'indirizzo fisico richiesto;
- la dimensione del segmento, riportata nella TDS, viene anche utilizzata per controllare eventuali violazioni.

Segue un'immagine esplicativa riguardante la traduzione degli indirizzi:



Le dimensioni della TDS sono legate alle dimensioni dello spazio virtuale di indirizzamento di uj processo. I processori della famiglia Intel, supportano fino a 16000 segmenti di 64 kb. La TDS può essere costituita da qualche riga fino a migliaia di righe e viene tenuta in memoria in uno dei segmenti associati al processo.
Due registri hardware, **RBTDS** e **RLTDS**, contengono l'indirizzo di inizio e fine della TDS associata al processo in esecuzione e questo consente anche il controllo d'accesso a segmenti non assegnati al processo.

- **RBTDS**: Registro Base della Tabella dei Descrittori di Segmento;
- **RLTDS**: Registro Limite della Tabella dei Descrittori di Segmento.

Quindi, la base ed il limite della nuova TDS vengono caricati nei registri RBTDS e RLTDS. Quando un processo subisce uno swapping, al ritorno in memoria va aggiornata la sua TDS anche se generalmente si preferisce rigenerarla totalmente utilizzando la mappa statica fornita dal linker. L'eventuale compattazione richiede l'aggiornamento nella TDS delle righe relative ai segmenti spostati.

La traduzione di ciascun indirizzo bidimensionale virtuale richiede due accessi in memoria:

- uno alla TDS per tradurre l'indirizzo del segmento da virtuale a fisico;
- l'altro per accedere fisicamente alla locazione richiesta.

La segmentazione, quindi, raddoppia il tempo per l'accesso ad una locazione di memoria.
Se i descrittori di segmento sono pochi, o se alcuni sono utilizzati frequentemente, conviene specificare il loro caricamento in opportuni registri hardware. I processori della famiglia Intel, dispongono di 4 registri utilizzabili da schemi di segmentazione e sono:

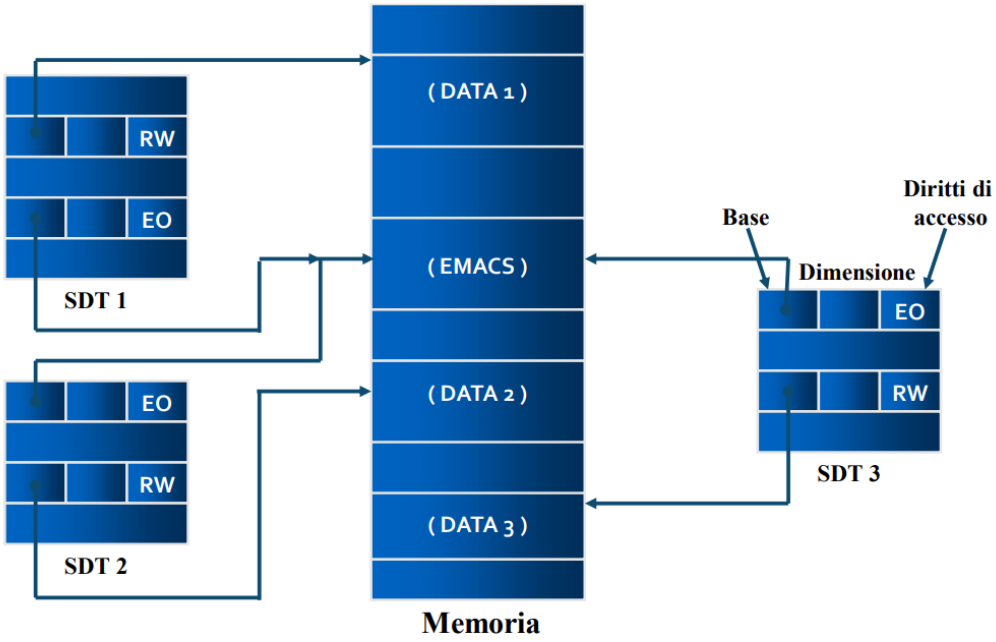
- **CS: Code Segment**;
- **SS: Stack Segment**;
- **DS: Data Segment**;
- **ES: Extra Segment**.

Meccanismi di protezione La segmentazione è lo schema migliore per quanto riguarda i meccanismi di protezione e anche di condivisione.
La forma più naturale di protezione per un sistema segmentato è l'utilizzo di registri base e limite. La protezione tra spazi di indirizzamento di processi differenti è garantita dalla collocazione dei segmenti in aree disgiunte di memoria. Una caratteristica offerta dai sistemi segmentati consiste nel fornire un livello di protezione all'interno dello spazio di indirizzamento di un singolo processo.
Infatti, è possibile definire diritti e modalità di accesso per ogni tipo di segmento. Ad esempio:

- accesso in modalità *execute* o *read-only* al segmento codice;
- accesso in modalità *read/write* al segmento stack;
- accesso *read-only* o *read/write* al segmento dati.

I diritti di accesso possono essere registrati in un campo della TDS.

Meccanismi di condivisione La segmentazione offre una condivisione semplice e flessibile. Gli oggetti condivisi sono collocati in segmenti dedicati e separati. Il segmento condiviso può essere mappato attraverso la TDS, nello spazio di indirizzamento virtuale di tutti i processo che hanno il permesso di accedervi e questo è possibile dato che lo spiazzamento interno di un dato è identico per tutti i processi che lo condividono dato che i segmenti iniziano con un indirizzo virtuale pari a 0.
Segue un esempio di meccanismi di condivisione tra processi:



Collegamento dinamico Il **collegamento dinamico** indica il caricamento di una procedura, come una libreria, in fase di esecuzione di un processo e solo su sua richiesta. Lo spazio in memoria per una procedura viene occupato solo quando questa viene richiesta. La segmentazione consente di aggiungere nuovi segmenti al processo richiedente, aggiornando la TDS e i registri base e limite della TDS.

Conclusioni La gestione della memoria fisica è più efficiente perchè non c'è più la necessità di allocare i processi in aree contigue. Inoltre, con la segmentazione la frammentazione interna è pari a 0, la frammentazione esterna

è ridotta al minimo, come detto in precedenza la segmentazione è lo schema migliore per quanto riguarda i meccanismi di condivisione e di protezione e inoltre, la segmentazione fornisce una funzionalità utile, cioè quella del **linking dinamico**.
Gli svantaggi principali che derivano dall'uso di questo schema sono diversi. Innanzitutto, vige la necessità di effettuare la compattazione che è più complessa. Inoltre, il duplice accesso alla memoria deve essere supportato da un hardware apposito, la gestione della memoria da parte del sistema operativo è più complessa rispetto a quella per il partizionamento statico e dinamico e non c'è ancora la possibilità di eseguire processi più grandi delle dimensioni fisiche della memoria.

4.4 Allocazione non contigua

4.4.1 Paginazione

Nella paginazione la memoria fisica viene concettualmente suddivisa in pagine di dimensione fissa chiamate **pagine fisiche** e anche lo spazio di indirizzamento dei processi viene diviso in **pagine virtuali** della stessa dimensione delle pagine fisiche.
Il meccanismo di traduzione fa corrispondere indirizzamenti all'interno di pagine virtuali ad indirizzamenti in pagine fisiche e ogni pagina è mappata separatamente quindi non è necessaria la contiguità delle pagine fisiche.
Nella paginazione la frammentazione esterna sparisce mentre la frammentazione interna è molto piccola che è al più un byte per processo.

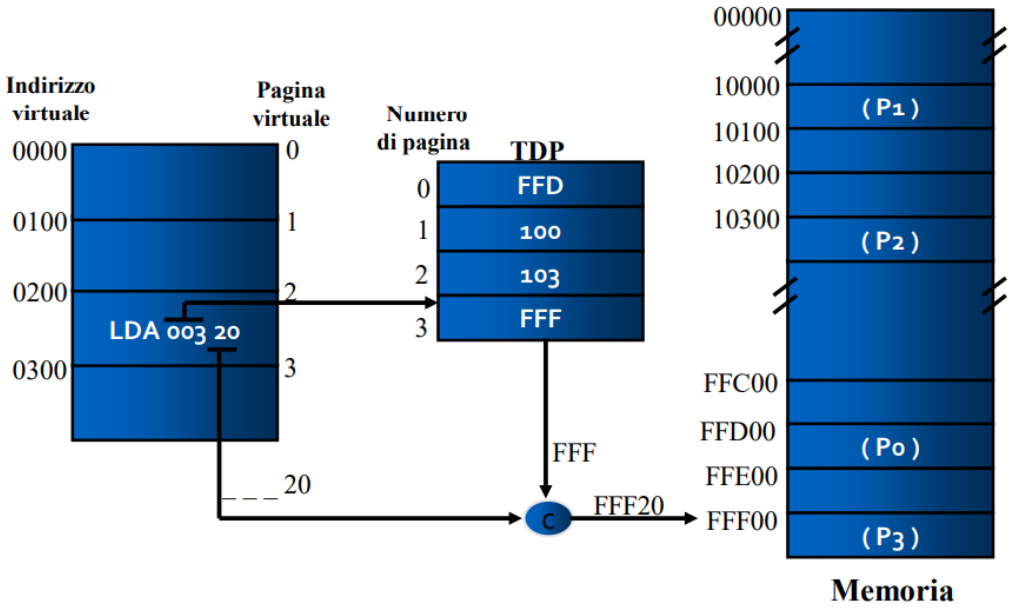
Principi operativi Ogni indirizzo all'interno di un processo è diviso in:

- bit più significativi che corrispondono al numeri di pagina virtuale;
- bit meno significativi che corrispondono allo spiazzamento reale.

Il sistema operativo tiene traccia della corrispondenza tra pagine virtuali e pagine fisiche mediante la **tabella delle pagine o TDP**. La TDP viene costruita al momento del caricamento del processo, ogni riga della tabella contiene l'indirizzo di partenza della pagina fisica in cui è allocata la corrispondente pagina virtuale e ne vengono salvati solo i bit più significativi e la tabella è costituita da un numero di righe pari al numero di pagine occupate.

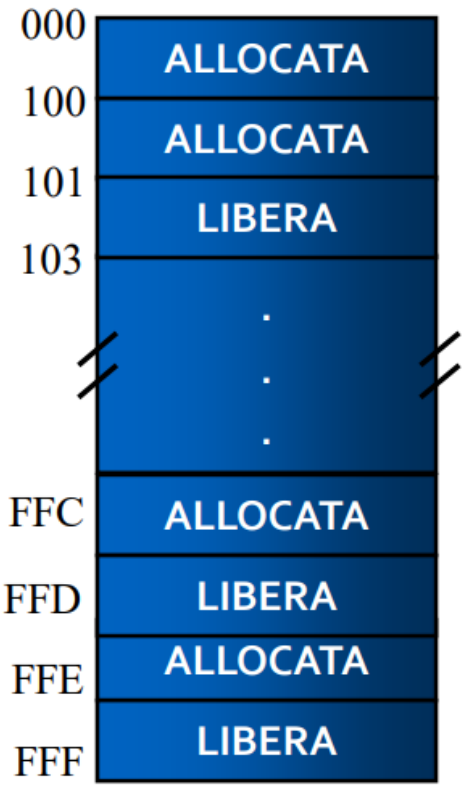
Ad esempio, supponiamo di avere una memoria di un 1 MB, suddivisa in 4096 pagine della dimensione di 256 byte ciascuna e con una lunghezza di indirizzo di 20 bit. Lo spazio di indirizzamento di un ipotetico processo utente che occupa 1008 byte (3F0H) viene suddiviso in quattro pagine virtuali da 0 a 3, in quanto quattro pagine permettono la memorizzazione di un processo di al massimo 1048 byte.
La **traduzione** dell'indirizzo virtuale avviene nel seguente modo:

- un meccanismo hardware divide l'indirizzo virtuale in:
  - 12 bit più significativi che indicano il numero di pagina;
  - 8 bit meno significativi che indicano lo spiazzamento all'interno della pagina.
- il numero di pagina viene usato all'interno della TDP per ottenere il corrispondente indirizzo della pagina fisica;
- l'indirizzo ottenuto, concatenato con il displacement, fornisce l'indirizzo fisico cercato.



Il sistema operativo tiene traccia dello stato di tutte le pagine fisiche mediante la tabella di memoria, la TDM:

- in ogni riga della TDM è indicato lo stato della relativa pagina se libera o allocata;
- il numero di righe della tabella è uguale al numero di pagine fisiche.



Quando un processo termina o subisce uno swapping:

- restituisce tutte le pagine fisiche ad esso assegnate;
- la sua TDP viene eliminata.

Quando il processo richiedente ha uno spazio di indirizzamento non multiplo delle pagine fisiche si crea il fenomeno della *frammentazione interna* della

pagina ma questo è un fenomeno ridotto e legato alla dimensione della pagina fisica.
Bisogna prestare attenzione, alla scelta della tecnica di **memorizzazione delle aree libere**, infatti, la consultazione della TDM statica per trovare n pagine fisiche libere richiede una ricerca su un numero di righe in media $x = \frac{n}{q}$, dove n corrisponde al numero di pagine libere che si vuole cercare e q è la probabilità che una pagina sia libera ed è legata alla percentuale di memoria libera u, da questo segue che $q = \frac{u}{100} \quad 0 < q < 1$, quindi x aumenta con l'aumentare della memoria utilizzata. Ad esempio:

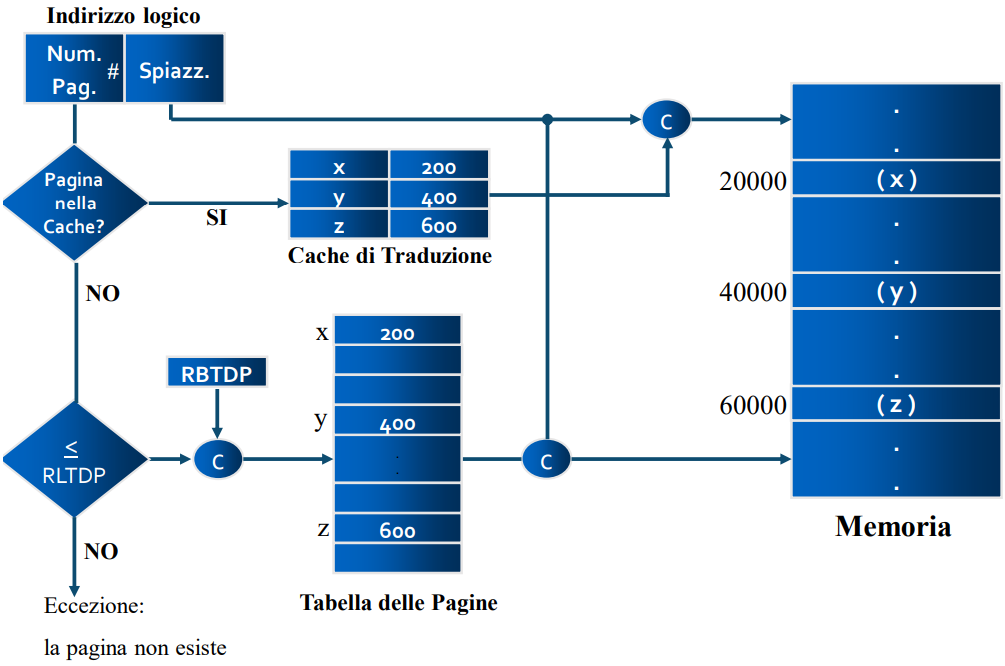
$$n = 10 \quad u = 40\% \quad q = 0.4$$

x in media è di 25 righe per ritrovare 10 pagine libere. Da qui deduciamo che la consultazione della TDM non è particolarmente efficiente, soprattutto nel caso in cui la memoria è satura e quindi si tende a sostituire la TDM con una lista a puntatori contenente i numeri delle pagine fisiche:

- n pagine da allocare si individuano nei primi n nodi della lista;
- in fase di disallocazione vengono inseriti in testa alla lista gli n nodi relativi alle pagine liberate.

Utilizzando questa tecnica il tempo di allocazione e disallocazione non dipende dalla percentuale di occupazione della memoria, però per il singolo elemento c'è un lieve spreco di memoria e di tempo rispetto alla gestione di una tabella statica.

Cache di traduzione :



Principi operativi I meccanismi hardware per la paginazione vengono utilizzati per due scopi:

1. **Risparmiare la memoria necessaria per le TDP**;
2. **Velocizzare la traduzione da indirizzi virtuali a fisici**.

1. Risparmiare la memoria necessaria per le TDP.
Durante la costruzione delle tabelle delle pagine è giusto pensare di costruirle con un numero di righe uguale alle pagine effettivamente utilizzate.
Di conseguenza, è necessario l'utilizzo di due registri hardware:

- registro **limite** della tabella TDP, **RLTDP** che contiene il numero di pagina virtuale più alto utilizzato dal processo;
- registro **base** della tabella TDP, **RBTDP** che contiene l'indirizzo di partenza della TDP.

I valori di questi due registri solitamente sono memorizzati nel descrittore di processo e questo meccanismo consente anche il controllo di accesso a pagine non assegnate al processo, vedi infatti l'esempio della cache di traduzione.

2. Velocizzazione del tempo di traduzione.
La traduzione di ciascun indirizzo virtuale richiede due accessi in memoria:

- uno alla TDP per prelevare il numero di pagina fisica;
- l'altro per accedere fisicamente alla locazione fisica.

Quindi la paginazione raddoppia il tempo per l'accesso ad una locazione di memoria e un metodo di velocizzazione è l'uso di una cache di traduzione che memorizza gli indirizzi delle pagine usate più frequentemente.

Meccanismi di protezione Nella paginazione la protezione interna viene meno rispetto alla segmentazione, questo perchè nella segmentazione era il programmatore a dividere il processo e stabiliva quali parti rendere solo leggibili o meno, mentre nella paginazione la divisione del programma viene effettuata dal sistema operativo e in una pagina è probabile trovare, ad esempio, pezzi di codice con pezzi di stack e quindi è difficile stabilire quale pagine rendere solo leggibile o meno.
Gli spazi di indirizzamento distinti vengono di norma allocati in zone disgiunte della memoria e gli accessi alla memoria sono ristretti al proprio spazio di indirizzamento.
Il registro limite della TDP blocca i tentativi di accedere a pagine virtuali oltre il limite assegnato al processo e i valori dei registri sono modificati esclusivamente da istruzioni privilegiate del sistema operativo.

Meccanismi di condivisione La condivisone è immediata con lo schema di paginazione, infatti:

- una copia unica di una pagina fisica può essere mappata in molti spazi di indirizzamento;
- i diversi processi possono avere un tipo di accesso diverso alla pagina;
- la condivisione viene riconosciuta ed effettuata da programmi di sistema, poichè la paginazione è trasparente all'utente;
- il codice condiviso deve essere naturalmente essere rientrante, cioè che non si automodifica e le variabili, costanti e stack sono raggruppati in aree separate. Un'alternativa al codice rientrante è di eseguire il codice condiviso in mutua esclusione

4.4.2 Memoria virtuale

La memoria virtuale viene anche chiamata paginazione su richiesta o segmentazione su richiesta quindi si rifa alle tecniche di gestione della memoria o della paginazione o della segmentazione.
La memoria virtuale è uno schema di gestione della memoria in cui soltanto una parte dello spazio di indirizzamento virtuale di un processo residente viene effettivamente caricata in memoria.
Con questo tipo di gestione si ha che:

- la somma di tutti gli spazi di indirizzamento virtuali dei processi attivi può superare la capacità della memoria fisica;
- la dimensione ammissibile per lo spazio di indirizzamento virtuale di un singolo processo può superare la capacità della memoria fisica disponibile di un sistema.

Gli schemi dei precedenti tipi di gestione della memoria tendevano ad una occupazione della memoria vicina al 100% mentre lo schema con memoria virtuale sembra fornire un grado di occupazione superiore al 100%.
Questo risultato si ottiene:

- mantenendo in memoria secondaria l'immagine dell'intero spazio virtuale di indirizzamento del processo;
- trasferendo in memoria fisica soltanto alcune parti al momento in cui sono necessarie;
- il sistema operativo sceglie tempi e modalità di trasferimento in memoria fisica tenendo conto di fattori quali:
  - richieste dei processi attivi;
  - priorità dei processi;
  - disponibilità globali del sistema.

Dal punto di vista del **programmatore**:

- i dettagli della gestione sono trasparenti;
- non è necessario l'adeguamento di un programma ad una memoria limitata;

- un programma può essere eseguito su sistemi con disponibilità di memoria fisica diversa, senza alcun adattamento.

Dal punto di vista del **sistema operativo:**

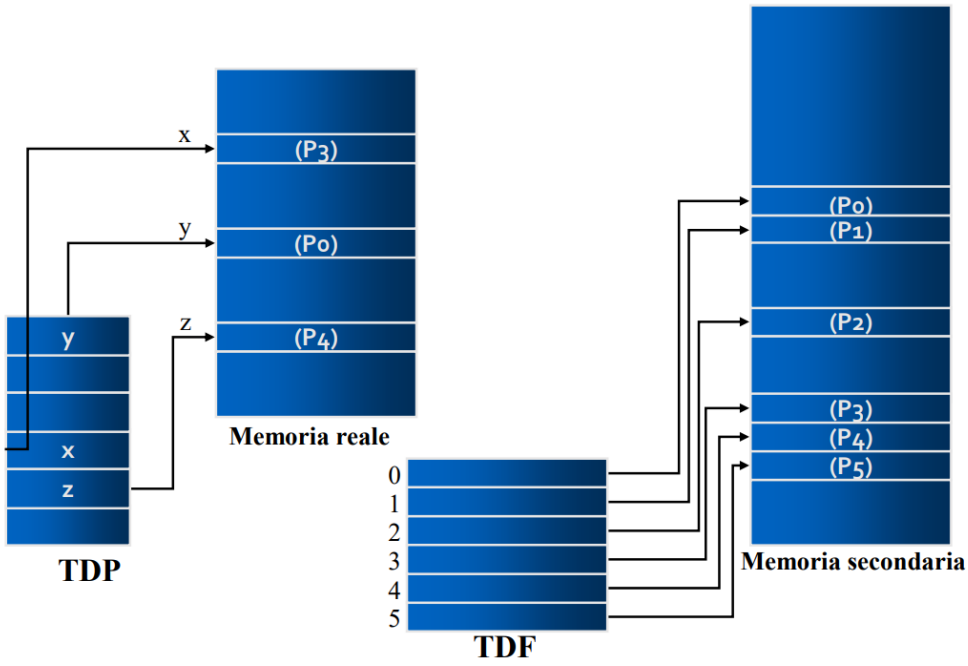
- un processo può essere caricato in uno spazio di dimensione arbitraria;
- si riduce la frammentazione esterna;
- non è necessario modificare l'ordine previsto di esecuzione di un processo;
- il quantitativo di memoria assegnato a un processo può variare durante la sua esecuzione;
- il sistema operativo può scegliere dinamicamente di velocizzare l'esecuzione di un processo allocando un quantitativo maggiore di memoria o, in alternativa, aumentare il grado di multiprogrammazione.

Bisogna fare alcune osservazioni. Un'istruzione può essere completata se quanto gli è necessario, cioè codice, stack e dati sono presenti in memoria. Se un processo richiede un oggetto fuori dalla memoria viene sospeso per un tempo relativamente lungo.
L'importanza dell'uso dello schema di memoria virtuale si basa su alcune osservazioni:

- durante una specifica esecuzione, alcune parti del programma non vengono indirizzate;
- le diverse condizioni interne o esterne causano diversi tracciati ad ogni esecuzione di un processo;
- alcune parti di un programma vengono eseguite molto raramente come la routine di gestione degli errori;
- alcuni programmi contengono tracciati complementari, come i compilatori.

Principi operativi Il funzionamento della memoria virtuale è analogo alla paginazione ed alla segmentazione e quindi con l'uso di tabelle di traduzione, la differenza che alcune pagine possono non esserci e quindi essere mancanti e quindi il processo di conversione è più complesso.
Il meccanismo di traduzione deve realizzare la funzione f che associa i nomi virtuali alle locazioni fisiche e deve restituire r se l'oggetto x è in memoria reale alla locazione r, in caso contrario deve restituire una condizione critica.

Tabelle Lo schema di memoria virtuale fa uso della tabella di traduzione delle pagine, nel caso di memoria virtuale con schema a paginazione, con aggiunta di 1 bit di presenza che segnala se questa è in memoria o 0 se è una pagina mancante, cioè una **page fault**.
Quando bisogna caricare un processo il sistema operativo crea la TDP, ponendo a 1 bit di presenza delle pagine portate in memoria. Se viene richiesta, successivamente, una pagina mancante il processo viene sospeso in attesa del suo caricamento e gli altri processi nel frattempo vengono eseguiti.
Lo schema di memoria virtuale fa uso anche della **tabella di descrizione del file, TDF** che associa al numeri di pagine virtuali l'indirizzo di residenza di queste sulla memoria secondaria.



Interrompibilità dell'istruzione In genere i sistemi supportano l'interruzione di un processo tra due istruzioni ma non supportano l'interrompibilità della singola istruzione e lo schema di memoria virtuale è implementabile solo in sistemi che supportano l'interrompabilità della singola istruzione nel caso in cui un'istruzione acceda ad una pagina non presente in memoria generando page fault.
L'assenza di memoria della pagina è un problema che deve essere gestito oltre all'interrompibilità dell'istruzione, quindi l'hardware deve essere in grado di gestire i cambiamenti avvenuti durante l'esecuzione della stessa.
Le possibili soluzioni a questo problema sono:

- **gli effetti parziali di un'istruzione vengono cancellati e l'istruzione viene ricominciata dopo aver risolto il page fault**, ma questo tipo di soluzione dipende strettamente dal tipo di istruzione e non è possibile generalizzare, vedi il caso dell'istruzione di stampa, alcuni cambiamenti sono irreversibili e può richiedere molto lavoro;

- **l'istruzione viene fatta partire dal punto esatto in cui era stata interrotta**, ma questo tipo di soluzione richiede forme sofisticate di memorizzazione dello stato di CPU che deve salvarsi lo stato di ogni singola istruzione che viene eseguita;

- **prima di iniziare l'esecuzione si controllano gli accessi in memoria che l'istruzione eseguirà**, però a volte può non essere possibili giustificare il controllo.

Le soluzioni che vengono adottate sono spesso miste e dipendono dall'architettura che si vuole ottenere.

Gestione della memoria virtuale La gestione della memoria virtuale segue gli schemi analoghi alla paginazione o alla segmentazione, con l'aggiunta della TDF. Ai fini dell'efficienza della memoria virtuale l'allocazione di un sottoinsieme di pagine virtuali su pagine fisiche richiede l'applicazione di particolari strategie che si possono classificare in:

- **strategie di allocazione:** quanta memoria reale allocare a ciascun processo;

- **strategie di ricerca:** quali oggetti trasferire e quando dalla memoria secondaria alla memoria centrale;

- **strategie di sostituzione:** in mancanza di spazio, quali oggetti rimuovere per assegnare spazio alle nuove richieste;
- **strategie di posizionamento:** dove collocare un oggetto che entra in memoria.

Strategie di allocazione e sostituzione Le strategie di allocazione e sostituzione sono immediate perchè gli oggetti vengono trasferiti quando servono infatti da questo deriva il nome di paginazione o segmentazione a richiesta; non sono facilmente applicabili strategie di previsione e il posizionamento delle pagine segue le regole dello schema con cui è realizzata la memoria virtuale, ovvero paginazione e segmentazione.
Queste due strategie sono molto importanti per l'efficienza della gestione della memoria virtuale e richiedono particolare attenzione. Analizzare il comportamento di un processo dal punto di vista delle sequenze di accesso alla memoria durante la sua esecuzione è molto importante al fine di:

- minimizzare il tempo di residenza in memoria di un processo, un processo viene sospeso in caso di page fault;
- minimizzare i trasferimenti di pagine perchè coinvolgono la CPU e i dispositivi di I/O.

Località del programmi Diversi studi hanno dimostrato che i programmi favoriscono un sottoinsieme del loro spazio di indirizzamento durante l'esecuzione. Questo fenomeno prende il nome di **località dei programmi** che è simile al concetto del principio di località della cache.
Dal punto di vista del sistema operativo la località si concretizza nel fatto che una parte degli accessi in memoria, in un periodo di tempo, vengono effettuati su un set ridotto della pagine virtuali di un processo, cioè un processo si muove lentamente da una località ad un'altra nel corso della sua esecuzione. Questa è una proprietà importante perchè è utilizzata per realizzare le strategie di allocazione e sostituzione delle pagine nello schema di gestione della memoria virtuale.

Strategie di sostituzione All'occorrenza di una page fault, se il gestore della memoria non ha più pagine disponibili può disporre di due operazioni:

- sospensione del processo che ha generato la page fault;

- sostituzione di una pagina in memoria.

La prima operazione viene scelta raramente in quanto l'eccezione di page fault è imputabile allo schema di gestione e non al processo e la sospensione del processo influirebbe sullo scheduling e sul tempo di risposta relativi al processo. Inoltre, in condizioni di memoria piena, il tempo che una pagina si liberi può essere lungo.
La seconda soluzione è quella che viene solitamente scelta perchè richiede preferibilmente la presenza di 1 bit per il controllo di pagina modificata e il dirty bit può rientrare tra i fattori presi in considerazione per scegliere la pagina da sostituire.

Algoritmi di sostituzione Esistono diversi algoritmi per la sostituzione delle pagine, quali:

- **algoritmo FIFO**;
- **algoritmo LRU (Least Recently Used)**;
- **algoritmo di Belady**;
- **algoritmo NRU (Not Recently Used)**.

Algoritmo di sostituzione - FIFO L'algoritmo FIFO sostituisce le pagine che da più tempo risiedono in memoria. Il gestore della memoria deve tenere traccia dell'ordine di caricamento delle pagine in memoria, ad esempio con una coda FIFO dei numeri di pagina. Questo algoritmo è molto semplice da realizzare, richiede di aggiornare la coda solo ad ogni page fault e non richiede supporti hardware specifici. Però, questo algoritmo spesso sostituisce pagine più frequentemente indirizzate, proprio perchè sono da più tempo in memoria e questo succede perchè questo algoritmo non tiene traccia dell'ordine di riferimento delle pagine.

Algoritmo di sostituzione - LRU L'algoritmo LRU sostituisce le pagine residenti che meno recentemente sono state utilizzate. Questo algoritmo considera la sequenza del comportamento di un programma con l'ipotesi che la pagina meno utilizzata recentemente è quella che ha la minore probabilità di essere referenziata in futuro.

Un possibile modo di operare è quello di memorizzare l'uso della pagine attraverso l'utilizzo di una struttura simile a uno stack: ogni volta che la pagine viene referenziata viene ricercata nello stack e posta in cima e la pagina da rimuovere viene presa dalla parte bassa dello stack.
Questo algoritmo si comporta meglio dell'algoritmo FIFO ma l'aggiornamento dello stack va effettuato ad ogni riferimento alla pagina e l'organizzazione di questo algoritmo richiede un supporto hardware dedicato alle operazioni relative all'aggiornamento dello stack.

ALgoritmo di sostituzione - Belady L'algoritmo di Belady sostituisce le pagine che saranno referenziate nel più lontano futuro. Notiamo, però, che questo algoritmo non è implementabile perchè non è possibile stabilire con certezza quali pagine verranno referenziate in un dato momento e quanto verranno referenziate. Tuttavia, questo algoritmo è molto importante a livello teorico perchè consente di confrontare le prestazioni degli altri algoritmi ad esecuzione finita. L'algoritmo di Belady è ottimale perchè minimizza il numero di **page fault** e questo ci permette di valutare quanto efficienti sono stati gli altri algoritmi di sostituzione che inevitabilmente genereranno più page fault a causa delle mancate informazioni future ma possiamo confrontare il numero di page fault ottenuto con il numero di page fault generato dall'algoritmo di Belady e capire se l'algoritmo scelto è stato efficiente o meno.

Algoritmo di sostituzione - NRU L'algoritmo NRU combina il basso costo FIFO con la registrazione delle pagine residenti tipico dell'LRU sostituendo le pagine non recentemente usate.
Funzionamento dell'algoritmo:

- Associa 1 bit a ciascuna pagina residente;
- il bit viene posto ad 1 ad ogni riferimento alla pagina;
- il bit di utte le pagine vengono posti a 0 ciclicamente;
- al momento di scegliere la pagine, l'algoritmo sceglie tra quelle con bit a 0;
- facendo uso di una lista circolare, riparte ogni volta dall'ultima pagina esaminata.

Algoritmi di sostituzione globale Gli algoritmi di sostituzione globale scelgono la pagina da rimuovere non solo tra quelle appartenenti al processo ma tra tutte quelle residenti. Questi algoritmi guardano più all'avanzamento globale del sistema che all'andamento individuale di ciascun processo e possono variare la strategia degli algoritmi di allocazione e questi algoritmi sono i meno utilizzati.

Strategia di allocazione delle pagine Le strategie di allocazione servono per decidere quanta memoria fisica allocare a ciascun processo attivo. L'allocazione di un numero pagine ottimale riduce la frequenza di page fault, riduce al minimo indispensabile il tempo di esecuzione e aumenta l'uso della CPU.
L'allocazione di un numero di pagine molto grande riduce il numero di processi attivi e abbassa il fattore di utilizzo della CPU.
L'allocazione di un numero di pagine troppo ridotto produce un aumento delle page fault, aumenta il tempo di residenza in memoria dei processi e aumenta lo spreco di CPU e dei dispositivi I/O per il trasferimento delle pagine.
Ad esempio, si consideri l'istruzione ADD [x], [y] assumendo che l'OP code e gli operandi siano codificati ciascuno in una parola, questa istruzione richiede:

- 3 accessi in memoria per leggere l'istruzione;
- 2 per leggere gli operandi;
- 2 per leggere i valori finali da sommare;
- 1 per scrivere il risultato.

Nel caso peggiore, l'esecuzione di questa istruzione richiede 6 pagine differenti e nel caso in cui al processo sono state allocate meno di 6 pagine si avrà che il processo non completerà mai questa istruzione e provocherà continui page fault.
Esistono delle indicazioni sul numero minimo di pagine da assegnare a un processo e riguardo al numero massimo si considera:

- relazione tra frequenza di errori di pagina e memoria assegnata;
- i parametri dipendono dal particolare programma, ma l'andamento degli errori di pagina è simile.

Un buon algoritmo di allocazione deve variare dinamicamente il numero di pagine allocate ad un processo, in base alla sue richieste ed allo stato del sistema tenendo presente:

- il limite *inferiore:* sotto il quale l'aumento degli errori di pagina aumenta molto rapidamente;
- il limite *superiore*: oltre il quale l'allocazione di altre pagine produce un modesto miglioramento delle prestazioni

Algoritmi troppo semplificati possono provocare il *trashing*, cioè l'eccessivo scambio di pagine di dati tra la RAM e la memoria secondaria.

Working set Un orientamento comune nella gestione della memoria è che le strategie di allocazione e sostituzione non possono essere trattate in modo indipendente in quanto le strategie di allocazione determinano le condizioni operative delle strategie di sostituzione e le strategie di sostituzione possono variare il quantitativo di memoria allocata ai processi.
La teoria del Working Set tratta in maniera interattiva l'allocazione e la sostituzione indicando quante pagine assegnare ad un processo e quali pagine è necessario tenere in memoria.
Le premesse su cui si poggia la teoria del Working Set sono:

- durante qualsiasi intervallo di tempo, un processo in esecuzione favorisce un sottoinsieme delle proprie pagine;
- la sequenza d'accesso alla memoria da parte di un processo mostra una relazione tra il passato immediato e il futuro;
- la frequenza con cui una pagina viene referenziata è una funzione variante lentamente nel tempo.

Il working-set $W(t, \Delta)$ viene definito come l'insieme delle pagine referenziate da un programma durante un recente intervallo di tempo:

$$W(t, \Delta) = \{i \in \mathbb{N} \mid \text{la pagina } i \text{ si trova tra } r_{t-\Delta+1} \text{ ed } r_t\}$$

dove r_t denota l'accesso in memoria all'istante t.
La teoria del working-set suggerisce che:

- un programma deve essere in esecuzione se e solo il suo working-set è in memoria;

- una pagina di memoria può essere rimpiazzata solo se non fa parte del suo working-set.

La determinazione accurata del working-set richiede l'intervento del sistema operativo dopo ogni accesso in memoria, pertanto la maggior parte delle realizzazioni pratiche approssimano la teoria del working-set.

Supporti hardware I supporti hardware richiesti sono gli stesso dello schema di gestione che realizza la memoria virtuale, segmentazione o paginazione in più la memoria virtuale richiede supporti per l'aggiornamento di:

- 1 bit di presenza;
- 1 bit d'uso.

Bisogna porre particolare attenzione nella scelta della dimensione della pagina in relazione alla dimensione di allocazione dei blocchi sulla memoria secondaria.
La protezione e la condivisione in un sistema a memoria virtuale mantengono le caratteristiche dello schema con cui è realizzata la memoria virtuale.
La rimozione della memoria di un oggetto condiviso comporta un aggravio tale di operazione da compiere e quindi si preferisce semplificarne la gestione lasciandolo in memoria.

Conclusioni Dal punto di vista dell'utente la memoria virtuale consente ai programmatori di concentrarsi sulle prestazioni dei loro programmi senza preoccuparsi del quantitativo di memoria richiesta e lo stesso programma può essere eseguito su elaboratori con memoria fisica diversa.

Capitolo 5

File system

Le applicazioni su un calcolatore hanno bisogno di memorizzare e rintracciare informazioni. Un processo può utilizzare il suo spazio di indirizzamento per memorizzare un quantitativo **limitato** di informazioni, però, può essere necessario che le informazioni siano memorizzate per lungo tempo e le informazioni nello spazio degli indirizzi di un processo vengono perse al termine dell'esecuzione del processo.
Inoltre, più processi possono aver bisogno delle stesse informazioni contemporaneamente per cui è necessario che queste e la loro allocazione siano indipendenti dal processo.
Per memorizzare grandi quantità di informazioni, rendere la memoria permanente e dare la possibilità a più processi di accedere alla stesse informazioni è necessario registrare tali informazioni in dischi o altri supporti in unità dette **file**.
Il sistema operativo consente la gestione dei file mediante il modulo di gestione del **File System**. Oltre al contenuto del file, il file system gestisce informazioni complementari relative al file come:

- struttura;
- nome;
- tipo di accesso;
- protezione, ecc.

I file consentono di ritrovare le informazioni che contengono mediante il proprio nome sul supporto in cui risiedono. Il nome del file si suddivide

generalmente in:

$$Nome.Estensione$$

entrambi con un numero variabile di caratteri e varia utilizzazione del minuscolo/maiuscolo. Spesso l'estensione indica un particolare tipo di file, come .txt, .c, .lib, ecc.

5.1 Struttura all'interno dei file

I file in genere, contengono le informazioni in tre tipi di strutture:

- **sequenza non strutturata di byte**: consente la massima flessibilità al programma utente. Sistemi operativi molto diffusi come DOS e Unix utilizzano questa struttura;
- **sequenza di record a lunghezza fissa**: ha perso popolarità col tempo;
- **albero di record con campo chiave**: è ancora in uso sui grossi mainframe usati per l'elaborazione di dati commerciali.

5.2 Tipologie di file

Nel corso del tempo il modello file, per la sua comodità d'uso, si è articolato in diversi tipi di file:

- **file regolari**: contengono le informazioni dell'utente, un programma eseguibile ecc.
- **directory**: conservano la struttura del file system ed informazioni sui file regolari;
- **file speciali a caratteri**: sono usati per modellare unità di input/output seriali come video, stampanti, reti, ecc;
- **file speciali a blocchi**: sono usati per modellare unità di input/output a blocchi come i dischi.

5.3 Formati dei file

Le informazioni nei file sono generalmente contenute in due formati:

1. **codice ASCII:** rappresenta caratteri quasi tutti leggibili e i file programma sorgente ed i documenti sono generalmente registrati in questo formato;

2. **codice Binario:** le sequenze di codici sono illeggibili e di solito i file memorizzati in questa modalità hanno una struttura interna.

5.4 Accesso ai file

Le modalità di lettura delle informazioni in un file sono di due tipi:

- **Accesso sequenziale**.
  I record sono letti uno dopo l'altro, dall'inizio alla fine e i vecchi sistemi operativi consentono solo questo tipo di accesso. Nei file speciali a caratteri si accede in questo modo;

- **Accesso casuale**.
  L'accesso al blocco di informazioni avviene in maniera diretta e l'avvento dei dischi ha consentito l'introduzione di questo metodo di accesso e generalmente l'accesso è diretto al blocco e sequenziale all'interno del blocco.

5.5 Attributi dei file

I sistemi operativi associano ai file altre informazioni come:

- data e ora della creazione;

- proprietario;

- dimensione, ecc.

Queste informazioni si definiscono *attributi* dei file.

Campo	Significato
Protezione	Chi può accedere al file e in che modo
Password	Parola d'ordine necessaria per accedere al file
Flag di sola lettura	0 per lettura/scrittura, 1 per sola lettura
Flag di sistema	0 per file normale, 1 per file di sistema
Flag ASCII/binario	0 per file ASCII, 1 per file binario
Lunghezza record	Numero di byte in un record
Tempo di creazione	Data e ora del momento in cui è stato creato il file
Dimensione attuale	Numero di byte nel file
Dimensione massima	Dimensione massima che il file può raggiungere

5.6 Operazioni sui file

Il sistema operativo pone a disposizione dell'utente *comandi di alto livello* e *chiamate di sistema* per la gestione dei file.
Alcune chiamate più comuni sono:

- **create**: creazione del file;
- **delete**: cancellazione del file;
- **open**: apertura del file con varia modalità;
- **close**: chiusura del file;
- **read**: lettura di blocchi di dati;
- **write**: scrittura di blocchi di dati;
- **append**: aggiunta, in coda al file, di nuovi dati;
- **seek**: posizionamento del puntatore all'interno del file.

5.7 File mappati in memoria

Alcuni sistemi operativi, come Unix, consentono di **mappare** un file in memoria, ossia di caricarlo tutto o in parte, assegnandogli degli indirizzi virtuali. Questo permette ai processi di accedere in maniera trasparente ai file che possono essere contenuti in memoria centrale per intero o per blocchi.
Questa tecnica velocizza le operazioni compiute sul file, soprattutto quando si riferiscono a parti limitate e non all'intero file. Quando il processo che sta usando il file termina, quest'ultimo viene riscritto sul disco.
La virtualizzazione dell'accesso comporta alcuni problemi che vanno gestiti:

- in caso di apertura contemporanea del file da parte di più processi alcune informazioni possono risultare inconsistenti;
- in caso di caduta del sistema alcune informazioni possono essere perse;
- sono necessarie tecniche per la rimozione delle pagine o dei blocchi dei file.

5.8 Directory

Per tenere traccia dei file, il file system fornisce un indice chiamato **directory**.
I vecchi sistemi prevedevano un'unica directory in cui confluivano tutti i file ma questo concetto è stato superato dai **sistemi gerarchici di directory**.
Tipicamente un elemento di directory contiene:

- il nome di un file ed eventualmente anche gli **attributi** del file;
- in alternativa, il nome di un file ed un **puntatore** ad un'altra struttura in cui si trovano gli attributi e gli indirizzi del disco.

Ogni directory può contenere:

- **file regolari**;
- altre **directory**

In questo modo, a partire da un directory **radice** o **root directory** è possibile generare un albero di directory e **subdirectory**. Il tipo di organizzazione dell'albero delle directory dipende dalla scelta dell'amministratore di sistema.
Un tipico esempio prevede:

- la root directory include alcune directory di sistema come *etc, bin, lib, tmp,* ecc;
- la root directory include una o più directory come *usr* per gli utenti;
- la directory *usr* contiene una subdirectory per ogni utente che organizza il proprio sottoalbero nel modo che ritiene più efficiente e comodo per il tipo di attività che svolge.

Quando il file system è organizzato come un albero di directory, è necessario un qualche modo per specificare i nomi dei file. Questo può essere fatto in due diversi modi:

- **path name assoluto**: consiste nel cammino dalla directory radice al file. I componenti del cammini sono separati dal simbolo "/" in Unix e "ïn DOS.
- **path name relativo**: si usa congiuntamente al concetto di directory di lavoro. Un utente può definire una directory come *directory di lavoro corrente*. In questo caso tutti i path name che non iniziano con la directory radice sono considerati relativi alla directory di lavoro.

Alcune tipiche chiamate di sistema per la gestione delle directory sono:

- **create**: crea una directory vuota;
- **delete**: cancella una directory;
- **opendir**: apre una directory per leggere il contenuto;
- **closedir**: chiude una directory;
- **readdir**: restituisce il prossimo elemento di una directory vuota;
- **rename**: rinomina una directory;
- **link**: crea un link tra un file esistente ed una pathname;
- **unlink**: rimuove un elemento dalla directory.

5.9 Implementazione del file system

Un fattore chiave nell'implementazione della memorizzazione dei file è tener traccia di quali blocchi del disco associare a ciascun file.
Le modalità di allocazione dei blocchi e del relativo reperimento sono:

- **Allocazione contigua**;
- **Allocazione a lista concatenata**;
- **Allocazione a lista concatenata con indice**;
- **Allocazione mediante uso di tabelle i-node**.

5.9.1 Allocazione contigua

Questo è lo schema di allocazione più semplice, in quanto memorizza il file in blocchi di disco consecutivi

Pregi Questo modalità di allocazione è molto semplice da implementare poichè tiene traccia del solo indirizzo del file memorizzando, quindi, un solo numero ed è molto efficiente perchè permette di leggere l'intero file con una sola operazione

Difetti Non è realizzabile senza che si conosca la dimensione massima del file e il disco risulta notevolmente frammentato e molto spazio viene sprecato.

5.9.2 Allocazione a lista concatenata

Ogni blocco associato al file contiene:

- dati veri e propri;
- il numero del blocco successivo assegnato al file.

Pregi Non implica spreco di spazio perchè i blocchi vengono allocati dinamicamente e nell'elemento delle directory viene memorizzato solo l'indirizzo del primo blocco.

Difetti La lettura sequenziale del file è semplice, ma l'accesso diretto ai blocchi è estremamente lento e la quantità di dati nel blocco non corrisponde più ad una potenza di due.

5.9.3 Allocazione a lista concatenata con indice

Assegna ai file:

- blocchi di disco non necessariamente contigui;
- mantiene l'elenco dei blocchi assegnati mediante una tabella in memoria.

Questo tipo di allocazione è utilizzata nel sistema operativo MS-DOS.

Pregi L'intero blocco è disponibile per i dati e l'accesso diretto ad un blocco è molto più rapido. Inoltre, l'elemento della directory contiene solo il numero del primo blocco.

Difetti Lo svantaggio principale consiste nel fatto che ad un gran numero di blocchi di disco corrisponde una tabella grande che occupa molto spazio in memoria centrale.

5.9.4 Allocazione mediante uso di tabelle i-node

Questo tipo di allocazione è utilizzata nel sistema operativo Unix.
Gli i-node sono piccole tabella che contengono:

- gli attributi del file;
- gli indirizzi dei primi dieci blocchi assegnati al file;
- tre campi che consentono l'indirizzamento dei blocchi successivi mediante:
  - blocco a singola indirezione;
  - blocco a doppia ed a tripla indirezione.

Gli i-node sono in posizione definita all'interno del file system.

Pregi L'elemento di directory contiene solo il nome del file ed il numero di i-node e consente di mantenere in memoria gli i-node dei file aperti sprecando poco spazio. Inoltre, consente di indirizzare un numero enorme di blocchi di disco e per i file di piccole dimensioni consente di reperire rapidamente l'indirizzo dei primi blocchi di disco.

5.10 Struttura delle directory

Il file system, mediante il *pathname* fornito dall'utente:

- localizza l'elemento della directory;
- estrae le informazioni necessarie per trovare i blocchi di disco assegnati al file.

5.11 Gestione dello spazio su disco

I file sono generalmente memorizzati su disco, pertanto la *gestione dello spazio su disco* è uno dei problemi fondamentali nell'implementazione del file system.
I file, generalmente, vengono memorizzati in blocchi di disco non necessariamente contigui. Questo perchè memorizzando un file con una sequenza contigua di byte si presenta l'ovvio problema che se il file aumenta di dimensione molto probabilmente dovrà essere spostato nel disco e lo spostamento di un file su un disco è un'operazione complessa e lenta.

5.11.1 Dimensione dei blocchi

Il primo problema da risolvere è la scelta della dimensione dei blocchi. La scelta di un blocco molto grande rende più veloce il tempo di lettura ma provoca maggiore di spazio, invece, se la dimensione del blocco è molto piccola causa numerosi accessi ai blocchi per leggere i dati ma consente una gestione più efficiente dello spazio.
L'efficienza nel tempo e nello spazio sono intrinsecamente in conflitto nella scelta delle unità di allocazione perchè ad una buona utilizzazione dello spazio corrisponde una bassa percentuale di dati e viceversa.
Un compromesso si raggiunge scegliendo la dimensione del blocco di allocazione di 512 byte, 1024 byte e 2048 byte.

Considerando che il *settore* è la minima unità fisica leggibile dal disco avremo che la dimensione del settore su disco è generalmente di 512 byte e la lettura di un blocco di allocazione di un 1Kb causerà sempre la lettura consecutiva di due settori di disco.

5.11.2 Gestione dei blocchi liberi

Il problema successivo alla scelta di dimensione del blocco è quello della gestione dei blocchi liberi.
I metodi comunemente usati sono due:

- **lista concatenata** di blocchi che contengono gli indirizzi dei blocchi liberi nel disco;
- **mappa di bit**: un disco di n blocchi richiede una mappa di n bit. I blocchi si rappresentano con 1 sulla mappa e 0 quelli allocati o viceversa.

Il metodo della lista concatenata è più efficiente se il disco è quasi pieno, poichè il numero di blocchi è molto limitato. Inoltre, in queste condizioni, la mappa di bit potrebbe risultare spesso piena e richiedere continui accessi ad altre parti di mappa per assegnare blocchi liberi. Se c'è abbastanza memoria principale per contenere la mappa di bit, è generalmente preferibile questo rispetto alla lista concatenata.

5.12 Affidabilità del file system

Il file system può contenere dati che non possono essere persi poichè di difficile o costoso recupero. L'affidabilità del file system va perseguita spesso anche a scapito dell'efficienza o di economie.

5.12.1 Gestione dei blocchi fuori uso

I dispositivi di memorizzazione presentano spesso blocchi inutilizzabili a causa di urti violenti, usura o difetti di fabbricazione. I blocchi inutilizzati non devono essere assegnati a nessun file e devono essere gestiti appropriatamente.
La soluzione per la gestione dei blocchi danneggiati possono essere di due tipi:

- **soluzioni hardware**: una parte di disco è riservata per tenere traccia dei blocchi rovinati e contiene dei blocchi di riserva. In fase di inizializzazione il controllore del disco legge questi blocchi e li sostituisce con blocchi di riserva, memorizzando l'avvenuta sostituzione. Questa soluzione è trasparente al file system.

- **soluzioni software**: l'utente o il file system tiene traccia dei blocchi rovinati assegnandoli tutti ad un file speciale da non utilizzare e nelle operazioni di backup è necessario porre attenzione per evitare le lettura di questo file.
  Ad esempio, il file system MS-DOS marca questi blocchi con un codice speciale 888 nella *File Allocation Table* e questi blocchi non verranno mai assegnati.

5.12.2 Copie di backup

Il metodo per ridurre le perdite di dati in caso di crash del file system è effettuare le copie di backup.
Nella pianificazione delle operazioni di backup vanno valutati principalmente:

- la frequenza con cui effettuare le copie di backup;
- i file che presentano la maggior necessità di essere copiati;
- la tecnica migliore per effettuare il backup.

Frequenza La scelta va pianificata in base alla quantità di lavoro svolta nel tempo ed al costo di ripristino. Generalmente la frequenza è di tipo:

- Giornaliera;
- Settimanale;
- Mensile;
- Annuale.

Scelta dei file La scelta va pianificata in base al contenuto dei file ed al costo di ripristino. Ad esempio:

- i file *critici* potrebbero essere raggruppati in directory dedicate delle quali si effettua copia con maggiore frequenza;
- la duplicazione di alcuni file *critici* può essere opportuna se il quantitativo di memoria sprecato è giustificabile.

Tecniche di backup La scelta va pianificata in base al tempo disponibile ed al costo sopportabile per i supporti. Le tecniche di backup possono essere:

- backup su nastri dell'intero file system;
- backup incrementale: copia solo dei file nuovi o modificati dall'ultimo backup;
- duplicazione incrociata dei dati su due dispositivi: metà di ogni disco contiene i propri dati e l'altra metà contiene i dati dell'altro disco.

5.12.3 Consistenza del file system

Nel corso della vita del file system può succedere che la sua consistenza venga a mancare a causa di crash, programmi che non rispettano le regole di allocazione o per altri motivi. La maggior parte dei sistemi operativi prevede un programma di servizio per la verifica della consistenza del file system e la correzione degli errori.
Ad esempio i controlli di consistenza di Unix sono di due tipi: sui blocchi e sui file.

Controllo di consistenza sui blocchi Nell'effettuare il controllo di consistenza sui blocchi Unix usa una tabella con due contatori per ogni blocco: il primo registra quante volte il blocco è presente in un file e il secondo quante volte è presente nella lista libera.
Il test consiste nella scansione degli i-node, per individuare i blocchi assegnati e la scansione della lista dei blocchi liberi. Al termine del check si possono verificare i seguenti casi:

- **caso normale**: un blocco è presenta una sola volta nella lista xor in un file;

- **blocco mancante**: un blocco non è presente nella lista libera e non è assegnato a nessun file, viene assegnato alla lista libera;
- **blocco duplicato nella lista libera**: il doppio riferimento viene cancellato;
- **blocco assegnato a due file:** è il caso più grave perchè è sintomo di possibili dati alterati. Il contenuto del blocco è copiato in un altro blocco ed ognuno dei blocchi assegnato ad un file;
- **blocco presente nella lista libera ed assegnato ad un file**: viene eliminato dalla lista libera.

Controllo di consistenza sui file Unix quando effettua il controllo di consistenza sui file tiene conto che file distribuiti su diverse directory possono puntare allo stesso file fisico, e quindi allo stesso i-node, in tal caso il contatore di i-node è > 1. Nel test l'albero delle directory viene interamente percorso e un contatore temporaneo per ogni i-node indica quanti file fanno riferimento ad ogni i-node.
I risultati che si possono verificare sono:

- **risultato normale**: contatore temporaneo è uguale al contatore di i-node;
- **contatore temporaneo $<$ contatore di link**: anche se tutti i file vengono rimossi, l'i-node rimane bloccato e al valore del link viene assegnato il valore del contatore.
- **contatore temporaneo $>$ contatore di link**: se alcuni file vengono rimossi, il valore del link si azzera. L'i-node viene reso libero e i blocchi ad esso assegnati vengono assegnati alla lista libera. Inoltre, potrebbero esserci file che puntano erroneamente all'i-node e i dati vengono persi e il contatore di link viene aggiornato al valore attuale.

5.13 Prestazioni del file system

L'accesso al disco è un'operazione notevolmente più lenta della lettura in memoria. Molti file system prevedono tecniche per ridurre il numero di accessi al disco come:

- **caching**;
- **riduzione dei movimenti del braccio del disco** (soluzione soft);
- **riduzione dei movimenti del braccio del disco** (soluzione hard).

5.13.1 Caching

Una delle tecniche più usate più comunemente è il *block cache* in cui un certo numero di blocchi in memoria vengono riservati per contenere blocchi di disco. La quantità dei blocchi riservati va scelta in funzione della memoria utilizzata e del miglioramento delle prestazioni richieste.
I principi operativi che la regolano sono:

- ad ogni richiesta di un blocco viene scandita la tabella di blocchi in memoria per verificarne la presenza;
- se non è presente il blocco viene caricato dal disco, copiato nella cache e poi reso disponibile alle richieste;
- la lista può essere gestita con normali algoritmi di rimpiazzamento delle pagine come FIFO o LRU;
- i blocchi critici vengono immediatamente riscritti sul disco quando modificati;
- per gli altri blocchi si può decidere di riscriverli subito oppure con una determinata frequenza.

5.13.2 Riduzione dei movimenti del braccio del disco, soluzione soft

La riduzione dei movimento del braccio del disco avviene nei seguenti passi:

- **raggruppamento di blocchi in uso**: andando a stimare i bocchi a cui si accede con maggiore probabilità in sequenza, al fine di ridurre i movimenti della testina. Si può osservare che con la gestione con mappa di bit è più facile scegliere un blocco libero vicino al precedente.

- **posizione dell'indice dei blocchi**. Tenendo conto che la lettura di un file richiede un duplice accesso, uno all'i-node ed uno al blocco in modo da ridurre la distanza fra gli i-node e i blocchi a cui questi fanno riferimento;

- **partizioni aperte**: si tende ad avvicinare gli i-node ai blocchi cui fanno riferimento andando a dividere il disco in gruppi di cilindri, ognuno con i propri i-node e la propria lista dei blocchi liberi e quando una partizione esaurisce i blocchi, un blocco può essere allocato in una partizione diversa;

- **interleave**: un processo richiede un tempo t_b per richiedere e recuperare un blocco. In situazioni di richieste consecutive un blocco richiesto può essere già passato sotto la testina e bisogna attendere una rotazione intera.
  La tecnica di interleaving alloca i blocchi fisici del disco in ordine non sequenziale stretto, ma sequenziale con salti.
  Sia t_r il tempo di rotazione del disco avremo che:

$$fattore\ di\ interleaving\ =\ \frac{t_r}{t_b} + 1$$

5.13.3 Riduzione dei movimenti del braccio del disco, soluzione hard

La riduzione dei movimenti del braccio del disco avviene mediante la *schedulazione del braccio del disco*. Il tempo necessario per leggere un blocco di disco dipende da:

- **seek time**: spostamento della testina sul cilindro;

- **latency time**: tempo di rotazione per raggiungere il settore richiesto;

- **transfer time**.

Il ritardo predominante è dato dal seek time, per cui vari algoritmi sono stati ideati per ottimizzare il movimento del braccio delle testine. Alcuni esempi sono:

- **FCFS**: serve le richieste nell'ordine in cui giungono e non richiede nessun specifico supporto hardware o software. Questo è l'algoritmo più

corretto nei riguardi delle richieste ma non ottimale perchè costringe il braccio del disco ad un superlavoro.

- **SSF (Shortest Seek First)**: mantenendo in tabella tutte le richieste pendenti, l'SSF si sposta sempre sulla richiesta più vicina a quella appena servita. Questo algoritmo riduce gli spostamenti sul disco ma presenta un problema di localizzazione delle richieste;

- **algoritmo dell'ascensore**: mantiene in tabella le richieste pendenti e presenta un bit up/down che indica la direzione attuale del braccio del disco e serve le richieste più vicine proseguendo sempre nella stessa direzione e cambia direzione quando non ci sono più richieste in quella direzione o quando raggiunge l'estremità. Questo algoritmo concilia meglio equità di servizio con efficienza.

5.14 Modelli di protezione

Gli utenti operano sugli archivi per mezzo dei processi di loro proprietà: le operazioni eseguibili dal generico processo sono limitate dai diritti di accesso attribuiti al suo proprietario. Le politiche che regolano l'attribuzione dei diritti d'accesso spetta al sistema operativo o all'utente medesimo.
Spetta, invece, al file system realizzare i meccanismi di protezione che impediscono agli utenti di superare i limiti dei propri diritti di accesso.

5.15 Dominio di protezione

Un modello generale di questi meccanismi è basato sul concetto di *dominio di protezione*. Un dominio di protezione è un insieme di coppie del tipo (Oggetto, DirittiDiAccesso).
Ad esempio:

- (F_i, {r,w}) esprime il possesso di lettura e di scrittura sull'archivio F_i.

Il modello basato sui domini consente di formalizzare il concetto della migrazione dei processi in un dominio di protezione diverso da quello di appartenenza. Per formalizzare la facoltà di migrare da un dominio all'altro è

sufficiente considerare anche i domini come oggetti ai quali è associata l'operazione di ingresso {in}. Una volta entrato in un nuovo dominio, l'utente o il processo proveniente da un altro dominio non ha la possibilità di migrare ulteriormente.

5.15.1 Strutture dati per la protezione

La matrice di protezione è tipicamente molto grande e sparsa. Una struttura più sufficiente è ottenuta rappresentando le stesse informazioni attraverso:

- **liste di controllo degli accessi**;
- **liste associate ai domini**.

Liste di controllo degli accessi Ogni lista è associata ad un oggetto ed i suoi elementi sono coppie del tipo (dominio, insieme dei diritti di accesso). Ciascuna lista elenca tutti i diritti di accesso sull'oggetto che competono ad un certo dominio. Queste liste solitamente sono riferite agli utenti che sono identificati con una coppia del tipo (gruppo, nome) dove i nomi sono locali ai gruppi. Ogni lista è un attributi degli archivi.

Liste associate ai domini In questo caso ogni lista è associata ad un dominio e i suoi elementi sono coppie del tipo (oggetto, insieme dei diritti di accesso) dove ogni elemento della lista esprime la facoltà dell'utente di accedere ad un dato oggetto con i diritti compresi in un certo insieme. Ogni lista è un attributo degli utenti e dei loro processi.

5.16 Protezione nel sistema operativo UNIX

I meccanismi di protezione adottati in UNIX si basano sulle liste di controllo degli accessi. Dato che ogni processo è attribuito ad un utente proprietario, l'identificazione degli utenti si propaga ai loro processi.
Ogni descrittore di archivio contiene una struttura dati simile, dal punto di vista del contenuto informativo ad una lista di controllo degli accessi. Essa consiste di tre codici:

- DirittiProprietario;

- DirittiGruppo;
- DirittiAltriUtenti.

che esprimono i diritti di lettura, scrittura ed esecuzione agli utenti proprietari, agli utenti del gruppo e agli utenti degli altri gruppi.
Questa convenzione non permette di discriminare tra utenti dello stesso gruppo del proprietario ne tra quelli appartenenti a gruppi diversi da quello del proprietario.

5.17 Identificazione degli utenti

I meccanismi di protezione degli archivi sono basati sul presupposto della sicura identificazione degli utenti.
Nei sistemi interattivi multi-utente, il progenitore di tutti i processi di un generico utente è quello che viene generato quando esso si connette al sistema attraverso un terminale.
Per ogni processo diverso dal progenitore, la responsabilità di definire il proprietario spetta al sistema operativo, che assegna il figlio allo stesso utente che risulta proprietario del processo padre. L'identificazione del proprietario del figlio è sicura se lo è quella del padre.
La sicurezza degli archivi è fondata sulla corretta definizione del proprietario del processo progenitore, che dipende dalla sicura identificazione dell'utente che si connette attraverso il terminale.
La tecnica comunemente usata è quella della password, che l'utente deve trasmettere attraverso il terminale, insieme al nome con il quale è registrato nel sistema, ogni volta che si connette.